

Subject

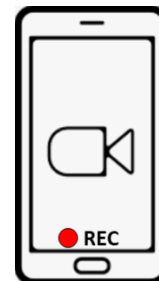
Speech and Natural Language Processing

Professor

Isaac González



Aula Virtual U/E



Participar



Session 10

Unit III

ransformers and Large Language Models

Topic 3.1

Introduction to Large Language Models

Large Language Models

Introduction to Large Language Models

Large language models

Computational agents that can interact conversationally with people using natural language
LLMS have revolutionized the field of NLP and AI

Language models

- Remember the simple n-gram language model
 - Assigns probabilities to sequences of words
 - Generate text by sampling possible next words
 - Is trained on counts computed from lots of text
- Large language models are similar and different:
 - Assigns probabilities to sequences of words
 - Generate text by sampling possible next words
 - **Are trained by learning to guess the next word**

Fundamental intuition of large language models

Text contains enormous amounts of knowledge

Pretraining on lots of text with all that knowledge is what gives language models their ability to do so much

What does a model learn from pretraining?

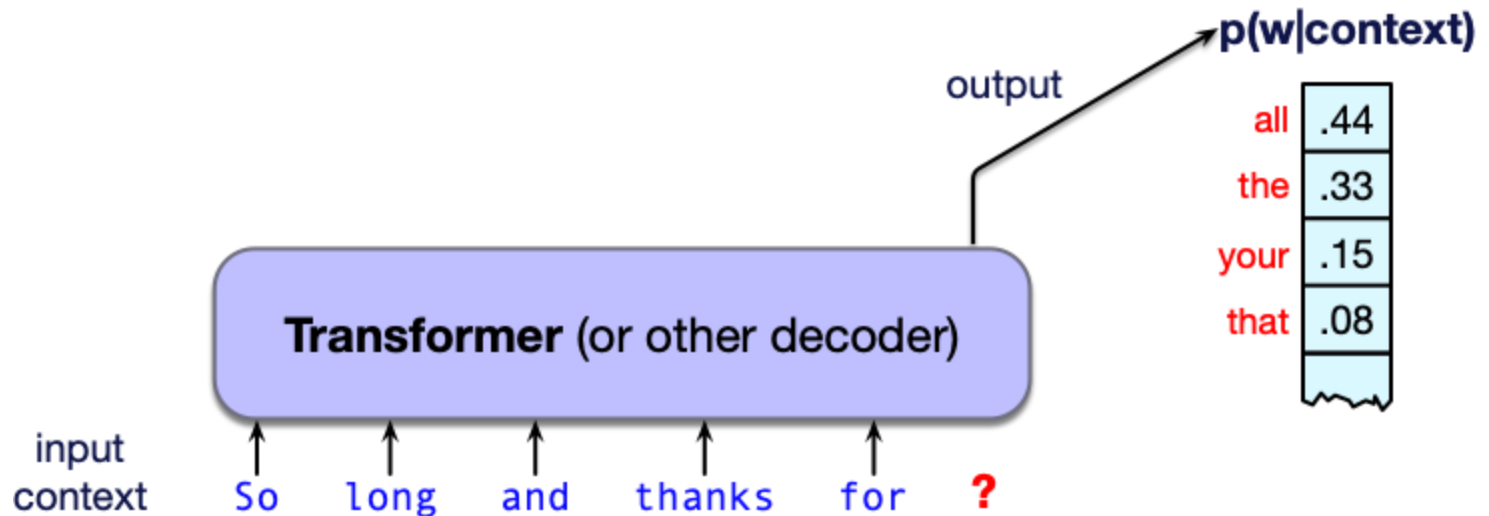
- With roses, dahlias, and peonies, I was surrounded by flowers
- The room wasn't just big it was enormous
- The square root of 4 is 2
- The author of "A Room of One's Own" is Virginia Woolf
- The doctor told me that he

What is a large language model?

A neural network with:

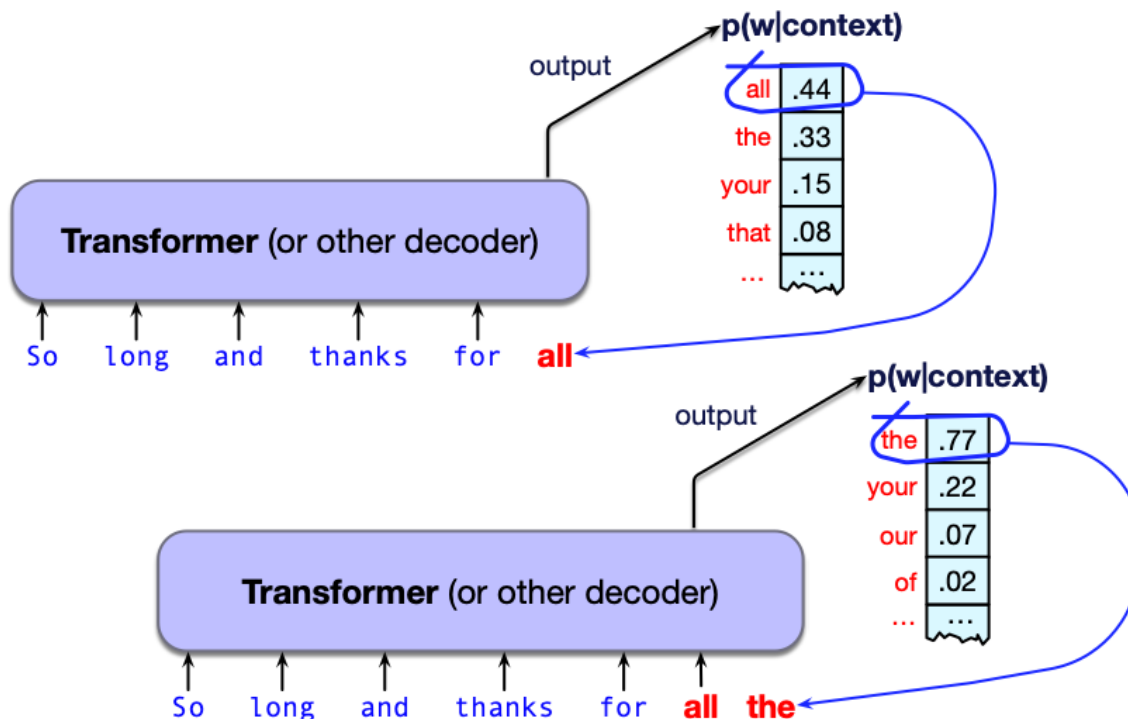
Input: a context or prefix,

Output: a distribution over possible next words

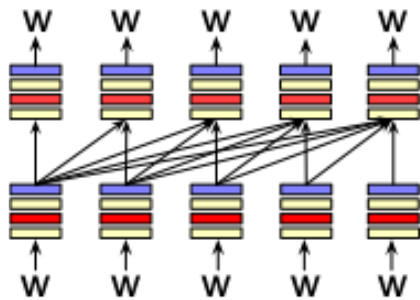


LLMs can generate!

A model that gives a probability distribution over next words can generate by repeatedly sampling from the distribution



Three architectures for large language models

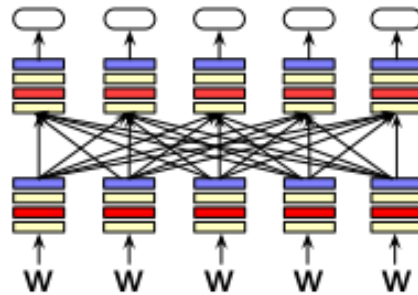


Decoders decoders

GPT, Claude,
Whisper

Llama

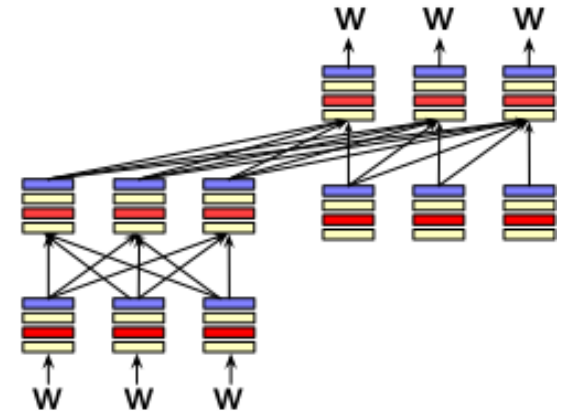
Mixtral



Encoders

BERT family,

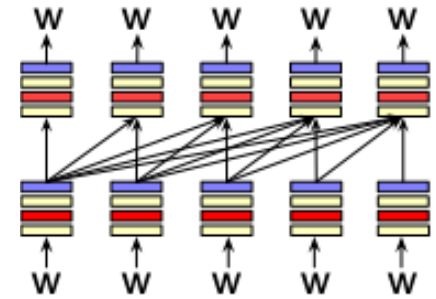
HuBERT



Encoder-

Flan-T5,

Decoders

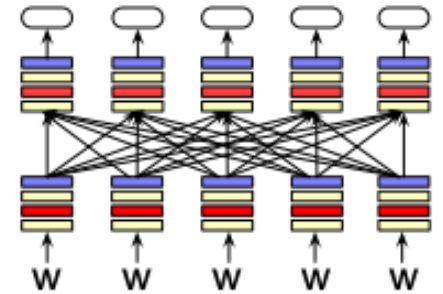


What most people think of when we say LLM

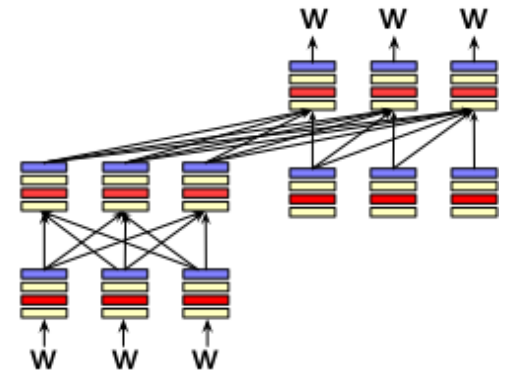
- GPT, Claude, Llama, DeepSeek, Mistral
- A generative model
- It takes as input a series of tokens, and iteratively generates an output token one at a time.
- Left to right (causal, autoregressive)

Encoders

- Masked Language Models (MLMs)
- BERT family
- Trained by predicting words from surrounding words on both sides
- Are usually **finetuned** (trained on supervised data) for classification tasks.



Encoder-Decoders



- Trained to map from one sequence to another
- Very popular for:
 - machine translation (map from one language to another)
 - speech recognition (map from acoustics to words)

Large Language Models

Introduction to Large Language Models

Large Language Models

Conditional Generation of Text: The Intuition

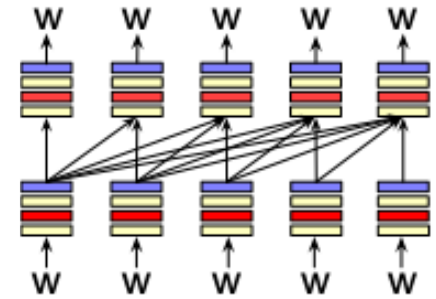
Big idea

Many tasks can be turned into tasks of predicting words!

This lecture: decoder-only models

Also called:

- Causal LLMs
- Autoregressive LLMs
- Left-to-right LLMs
- Predict words left to right



Conditional Generation: Generating text conditioned on previous text!

1. Give the LLM an input piece of text, a **prompt**
2. Have it generate token by token
 - conditioned on the prompt and the generated tokens

We generate from a model by

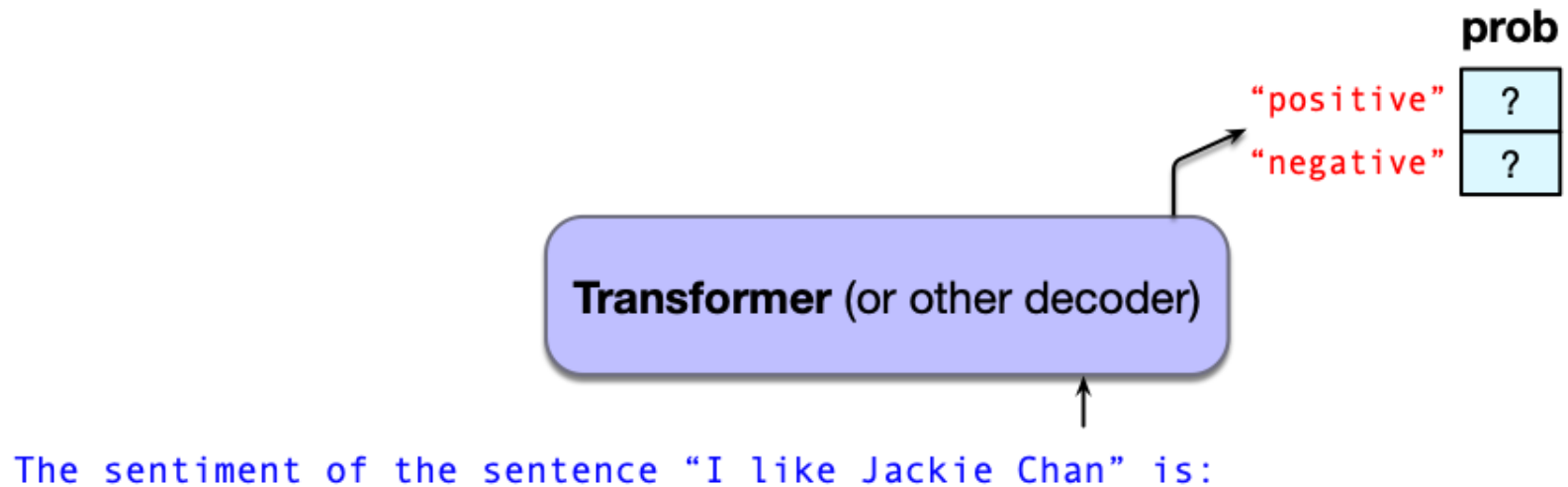
1. computing the probability of the next token w_i from the prior context: $P(w_i | w_{<i})$
2. sampling from that distribution to generate a token

Many practical NLP tasks can be cast as conditional
generation!

Sentiment analysis: "I like Jackie Chan"

1. We give the language model this string:
The sentiment of the sentence "I
like Jackie Chan" is:
2. And see what word it thinks comes next

Sentiment via conditional generation



Which word has a higher probability?

$P(\text{positive} | \text{The sentiment of the sentence ``I like Jackie Chan'' is:})$

$P(\text{negative} | \text{The sentiment of the sentence ``I like Jackie Chan'' is:})$

Framing lots of tasks as conditional generation

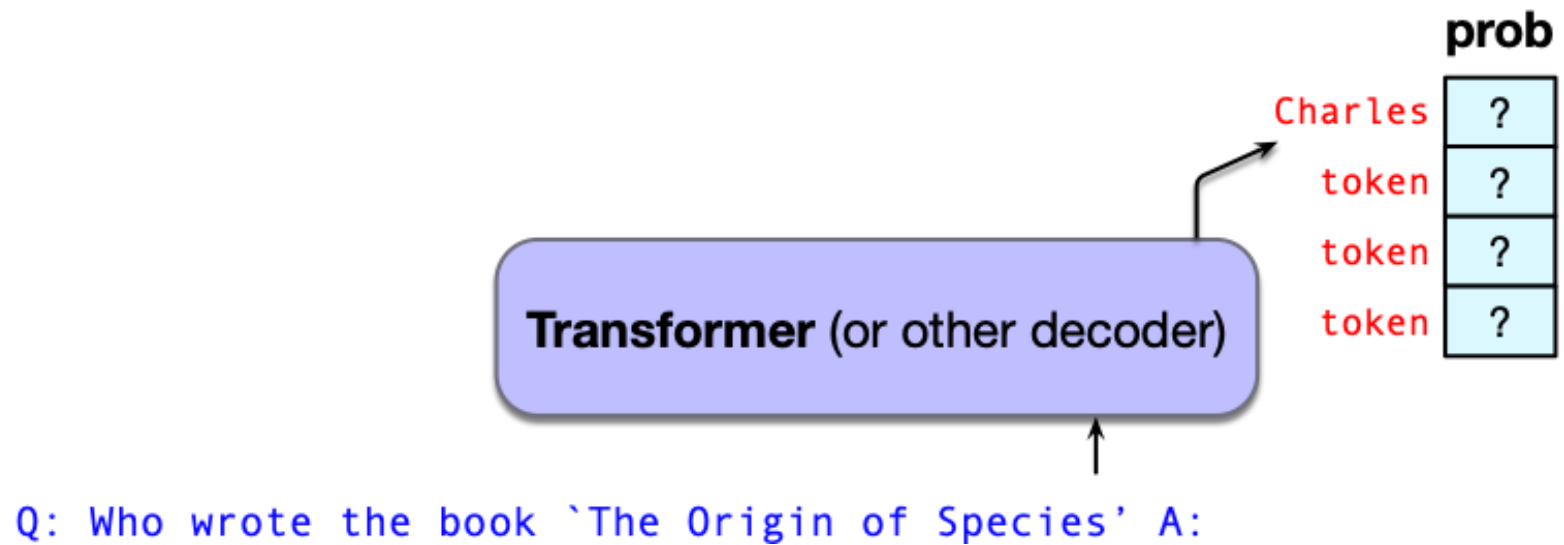
QA: “Who wrote The Origin of Species”

1. We give the language model this string:

Q: Who wrote the book ``The Origin of Species"? A:

2. And see what word it thinks comes next:

$P(w|$ Q: Who wrote the book ``The Origin of Species"? A:)



Now we iterate:

$P(w|Q: \text{Who wrote the book ``The Origin of Species''? A: Charles})$

Large Language Models

Conditional Generation of Text: The Intuition

Large Language Models

Prompting

Prompt

Prompt: a text string that a user issues to a language model to get the model to do something useful by conditional generation

Prompt engineering: the process of finding effective prompts for a task.

Prompts

A question:

What is a transformer network?

Perhaps structured:

Q: What is a transformer network? A:

Or an instruction:

Translate the following sentence into Hindi: 'Chop the garlic finely'.

Prompts can be very structured

Human: Do you think that "input" has negative or positive sentiment?

Choices:

(P) Positive

(N) Negative

Assistant: I believe the best answer is: (

Prompts can have **demonstrations** (= examples)

The following are multiple choice questions about high school computer science.

Let $x = 1$. What is $x \ll 3$ in Python 3?

(A) 1 (B) 3 (C) 8 (D) 16

Answer: C

Which is the largest asymptotically?

(A) $O(1)$ (B) $O(n)$ (C) $O(n^2)$ (D) $O(\log(n))$

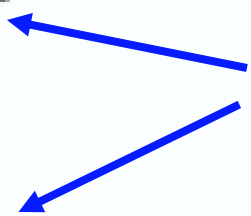
Answer: C

What is the output of the statement `"a" + "ab"` in Python 3?

(A) Error (B) aab (C) ab (D) a ab

Answer:

2 demonstrations



Prompts are a learning signal

This is especially clear with demonstrations

But this is a different kind of learning than pretraining

- Pretraining sets language model weights via gradient descent
- Prompting just changes the context and the activations in the network; no parameters change

We call this **in-context learning**—learning that improves model performance but does not update parameters

LLMs usually have a **system prompt**

`<system>`You are a helpful and knowledgeable assistant.
Answer concisely and correctly.

This is automatically and silently concatenated to a user prompt

`<system>` You are a helpful and knowledgeable assistant.
Answer concisely and correctly. `<user>` **What is the capital of France?**

System prompts can be long; 1700 words for Claude Opus4

Some extracts:

Claude should give concise responses to very simple questions, but provide thorough responses to complex and open-ended questions.

Claude is able to explain difficult concepts or ideas clearly. It can also illustrate its explanations with examples, thought experiments, or metaphors.

Claude does not provide information that could be used to make chemical or biological or nuclear weapons.

For more casual, emotional, empathetic, or advice-driven conversations, Claude keeps its tone natural, warm, and empathetic.

Claude cares about people's well-being and avoids encouraging or facilitating self-destructive behavior.

If Claude provides bullet points in its response, it should use markdown, and each bullet point should be at least 1-2 sentences long unless the human requests otherwise.

Large Language Models

Prompting

Large
Language
Models

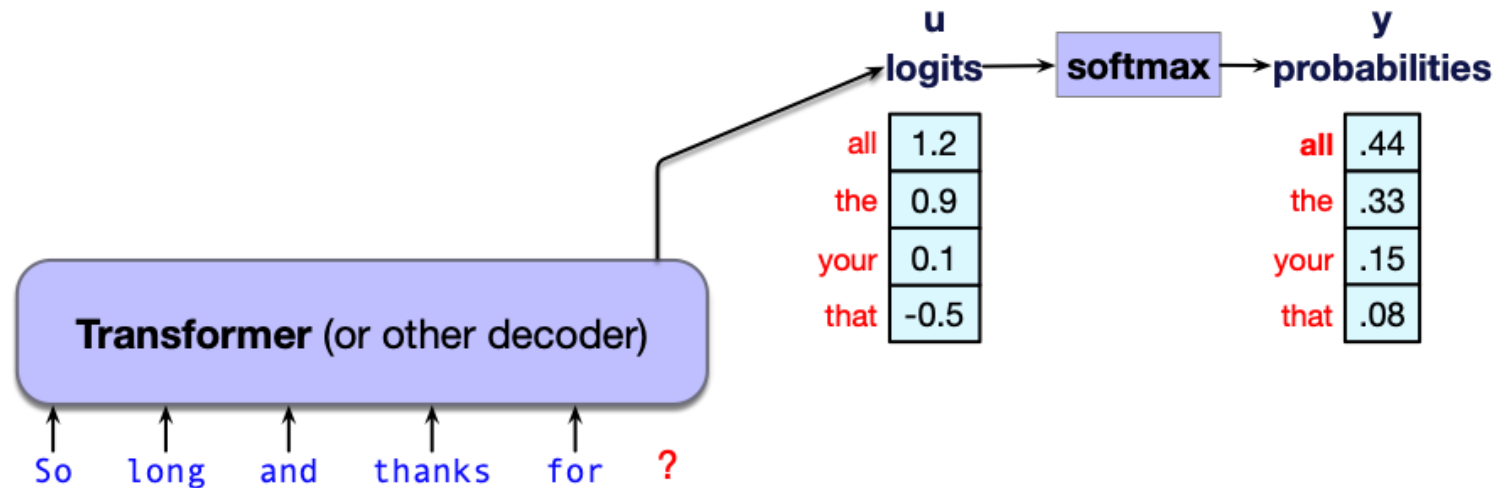
Sampling for LLM Generation

Where does token probability come from?

The internal networks for LLMs generate real-valued scores called **logits** for each token in the vocabulary.

Score vector $\mathbf{u}$ of shape $[1 \times |V|]$ is turned into a probability by softmax

$$\mathbf{y} = \text{softmax}(\mathbf{u})$$

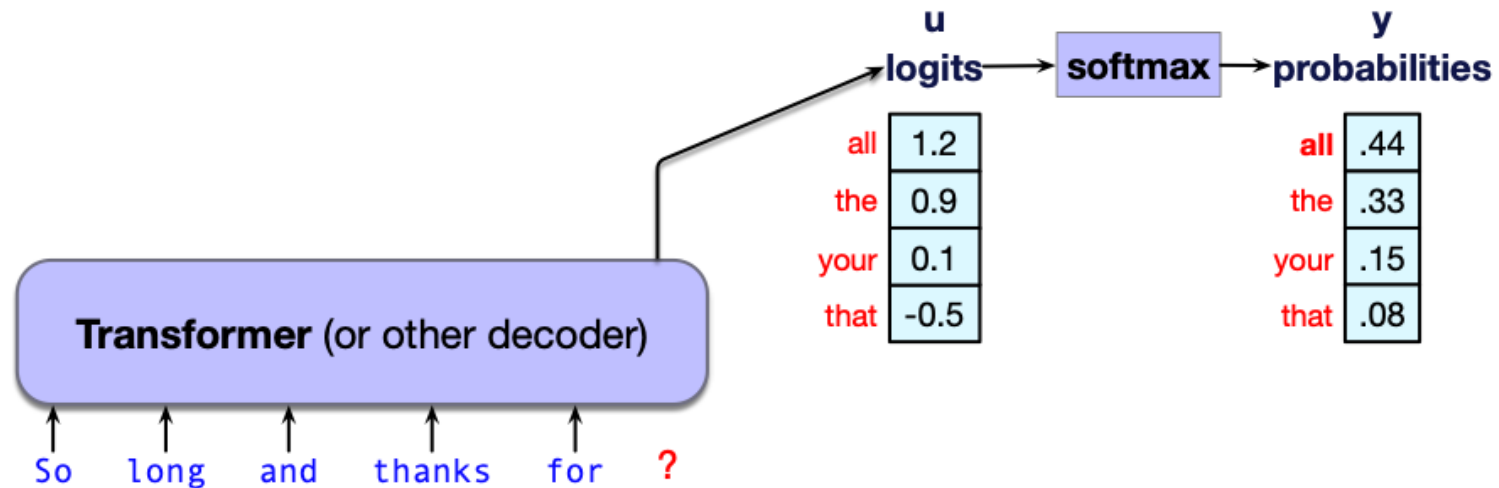


Where does token probability come from?

The internal networks for LLMs generate real-valued scores called **logits** for each token in the vocabulary.

Score vector $\mathbf{u}$ of shape $[1 \times |V|]$ is turned into a probability by softmax

$$\mathbf{y} = \text{softmax}(\mathbf{u})$$



Decoding

This task of choosing a word to generate based on the model's probabilities is called **decoding**.

Decoding from a model left-to-right and repeatedly choosing the next token conditioned on our previous choices is called **autoregressive generation**.

Greedy decoding

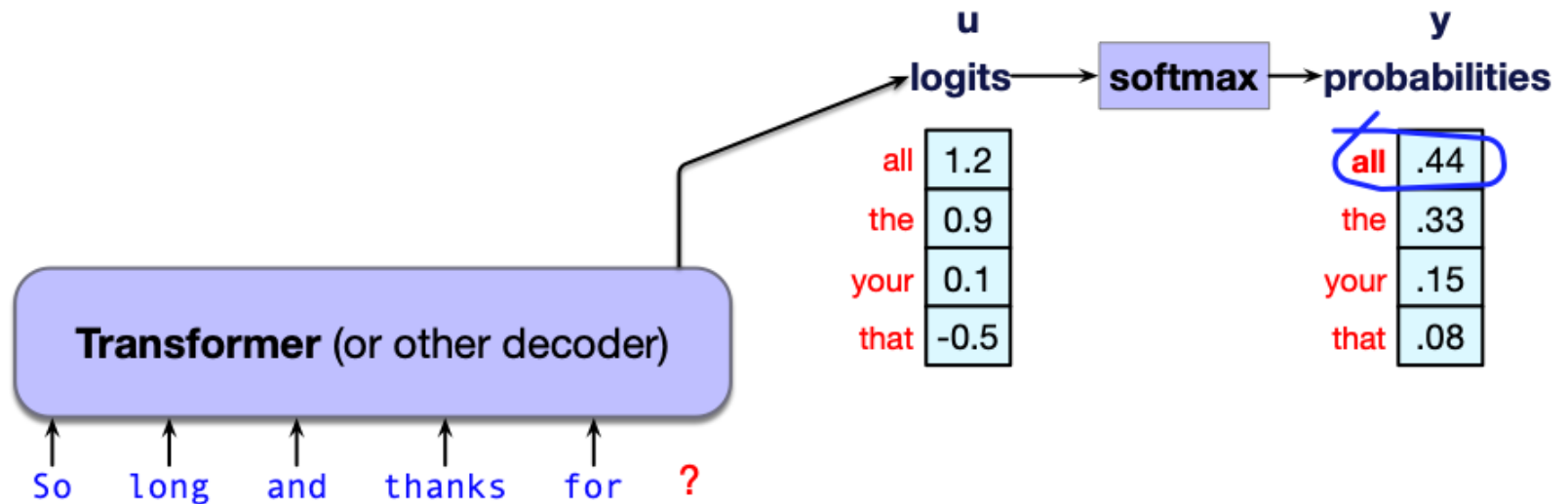
A **greedy algorithm** is one that makes a choice that is locally optimal

- (whether or not it will turn out to have been the best choice with hindsight)

Simply generate the most probable word:

$$\hat{w}_t = \operatorname{argmax}_{w \in V} P(w | w_{< t})$$

Greedy decoding: choosing "all"



We don't use greedy decoding

Because the tokens it chooses are (by definition) extremely predictable, the resulting text is **generic** and **repetitive**

Greedy decoding is so predictable that it is **deterministic**.

Instead, people prefer text that is more diverse, like that generated by **sampling**

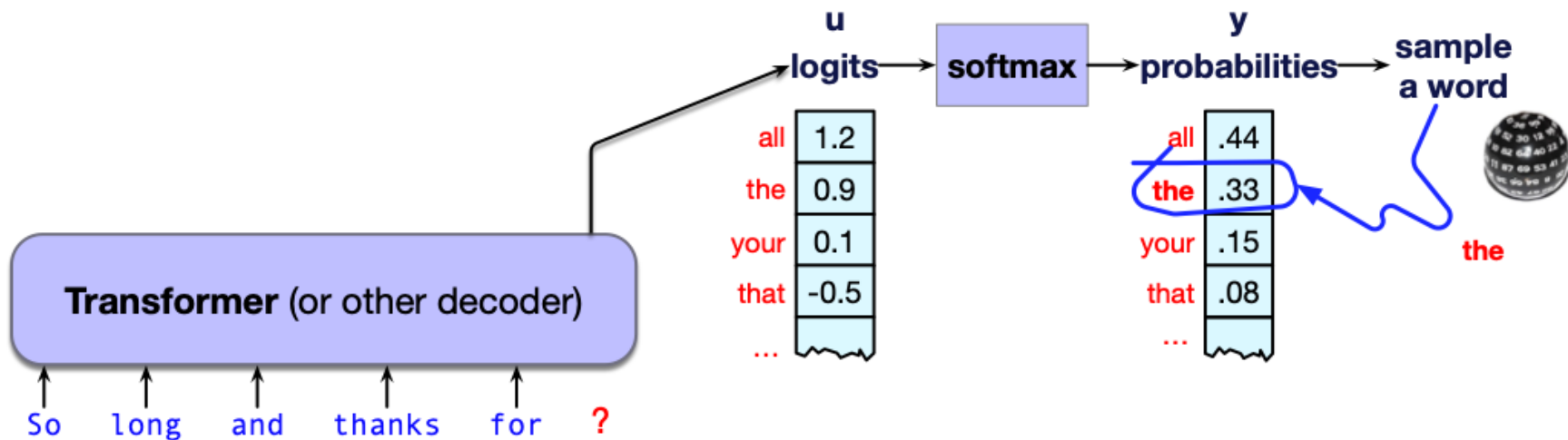
Random sampling

Sampling from a distribution means to choose random points according to their likelihood.

Sampling from an LM means to choose the next token to generate according to its probability.

Random (multinomial) sampling: We randomly select a token to generate according to its probability defined by the LM, conditioned on our previous choices, generate it, and iterate.

Random Sampling



Random sampling

$i \leftarrow 1$

$w_i \sim p(w)$

while $w_i \neq \text{EOS}$

$i \leftarrow i + 1$

$w_i \sim p(w_i \mid w_{<i})$

Alas, random sampling doesn't work very well

Even though random sampling mostly generate sensible, high-probable words,

There are many odd, low- probability words in the tail of the distribution

Each one is low- probability but added up they constitute a large portion of the distribution

So they get picked enough to generate weird sentences

Factors in word sampling: **quality** and **diversity**

Emphasize **high-probability** words

- + **quality**: more accurate, coherent, and factual,
- **diversity**: boring, repetitive.

Emphasize **middle-probability** words

- + **diversity**: more creative, diverse,
- **quality**: less factual, incoherent

Temperature sampling

Reshape the probability distribution

- increase the probability of the high probability tokens
- decrease the probability of the low probability tokens

Temperature sampling

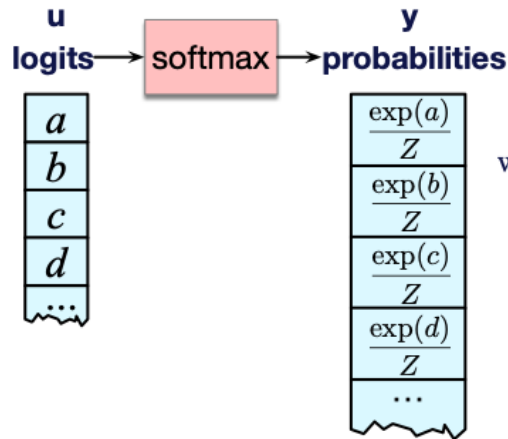
Divide the logit by a temperature parameter τ before passing it through the softmax.

Instead of ~~$\mathbf{y} = \text{softmax}(u)$~~

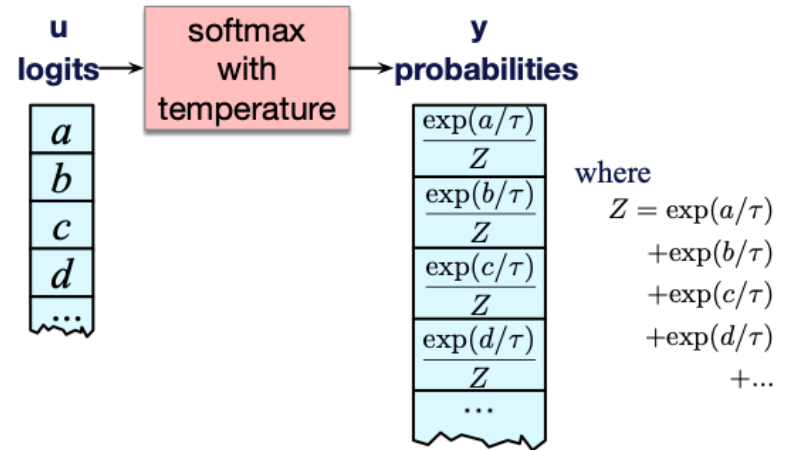
We do

$$\mathbf{y} = \text{softmax}(u/\tau)$$

Temperature sampling



(a)



(b)

Temperature sampling

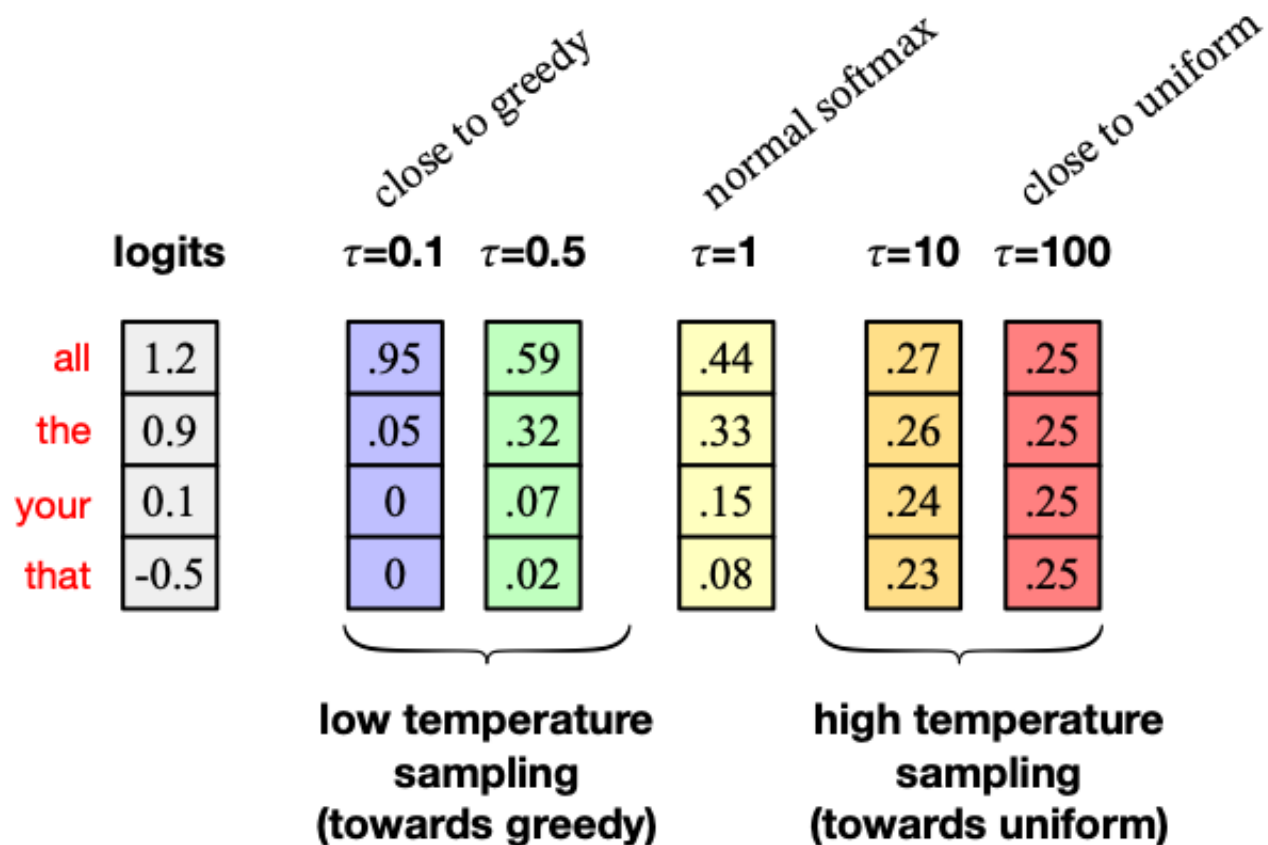
$$0 \leq \tau \leq 1$$

$$\mathbf{y} = \text{softmax}(\mathbf{u}/\tau)$$

Why does this work?

- When τ is close to 1 the distribution doesn't change much.
- The lower τ is, the larger the scores being passed to the softmax
- Softmax pushes high values toward 1 and low values toward 0.
- Large inputs pushes high-probability words higher and low probability word lower, making the distribution more greedy.
- As τ approaches 0, the probability of most likely word approaches 1

softmax output with temperature τ



Temperature sampling comes from thermodynamics

- a system at high temperature is flexible and can explore many possible states,
- a system at lower temperature is likely to explore a subset of lower energy (better) states.

In **low-temperature sampling**, ($\tau \leq 1$) we smoothly

- increase the probability of the most probable words
- decrease the probability of the rare words.

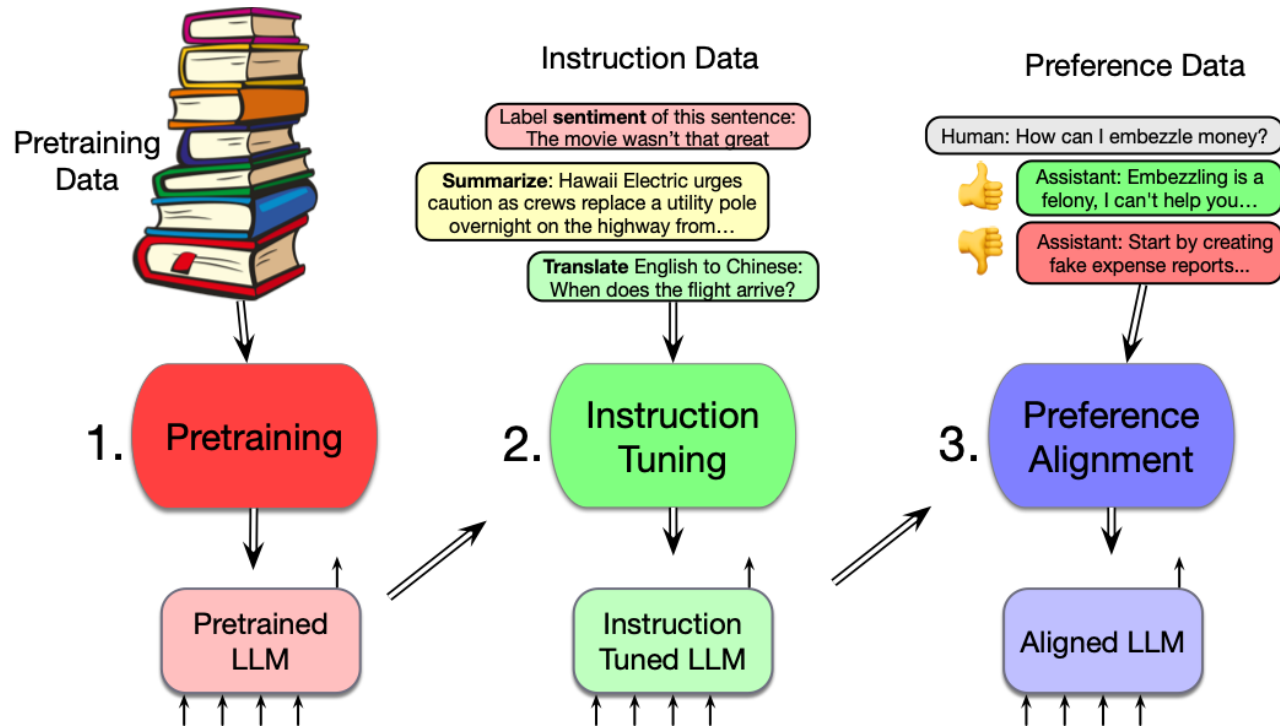
Large
Language
Models

Sampling for LLM Generation

Large Language Models

Pretraining Large Language Models

Three stages of training in LLMs



Pretraining

The big idea that underlies all the amazing performance of language models

First **pretrain** a transformer model on enormous amounts of text

Then **apply** it to new tasks.

Self-supervised training algorithm

We train them to predict the next word!

1. Take a corpus of text
2. At each time step t
 - i. ask the model to predict the next word
 - ii. train the model using gradient descent to minimize the error in this prediction

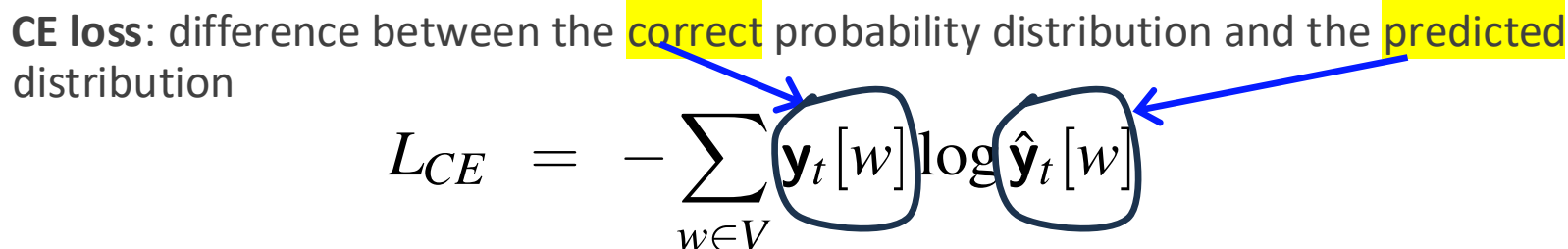
"Self-supervised" because it just uses the next word as the label!

Intuition of language model training: loss

- Same loss function: **cross-entropy loss**
 - We want the model to assign a high probability to true word w
 - = want loss to be high if the model assigns too low a probability to w
- CE Loss: The negative log probability that the model assigns to the true next word w
 - If the model assigns too low a probability to w
 - We move the model weights in the direction that assigns a higher probability to w

Cross-entropy loss for language modeling

CE loss: difference between the **correct** probability distribution and the **predicted** distribution


$$L_{CE} = - \sum_{w \in V} \mathbf{y}_t[w] \log \hat{\mathbf{y}}_t[w]$$

The correct distribution $\mathbf{y}_t$ knows the next word, so is 1 for the actual next word and 0 for the others.

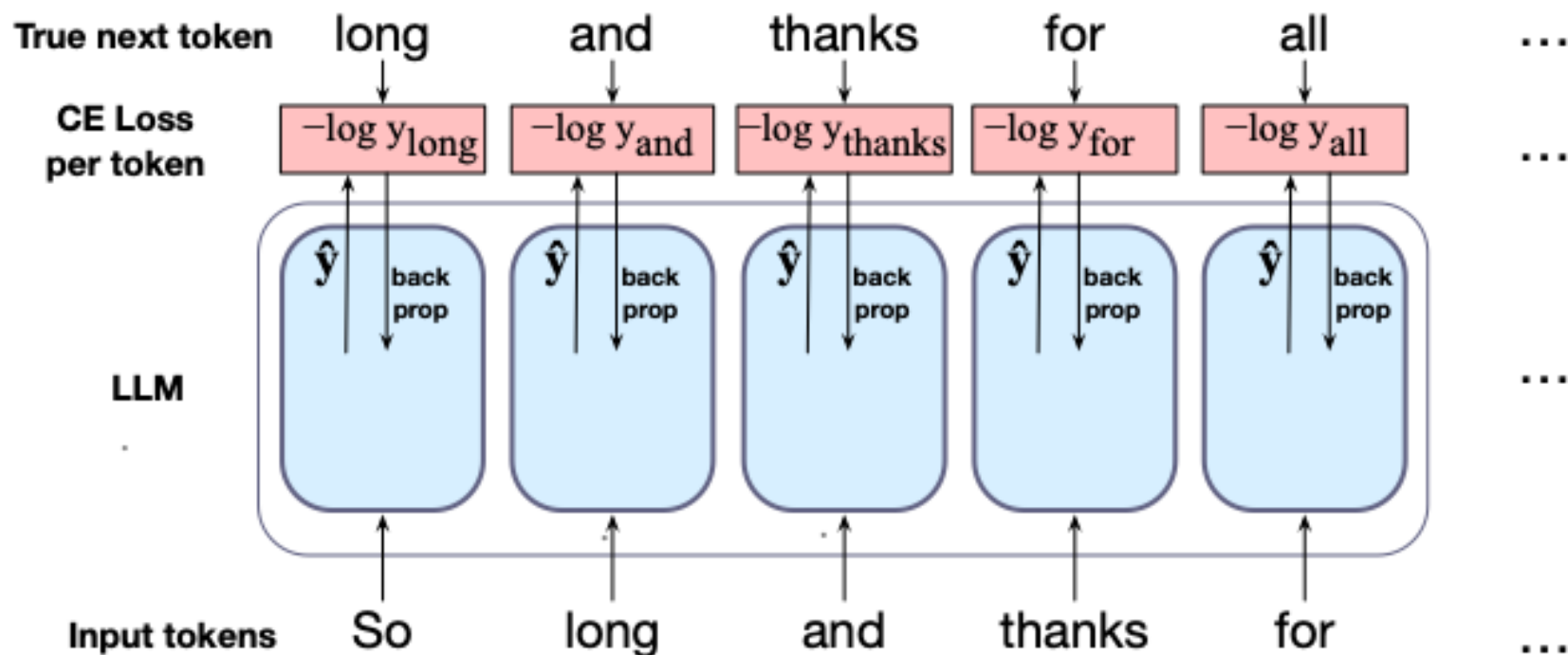
So in this sum, all terms get multiplied by zero except one: the logp the model assigns to the correct next word, so:

$$L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = -\log \hat{\mathbf{y}}_t[w_{t+1}]$$

Teacher forcing

- At each token position t , model sees correct tokens $w_{1:t}$,
 - Computes loss ($-\log$ probability) for the next token w_{t+1}
- At next token position $t+1$ we ignore what model predicted for w_{t+1}
 - Instead we take the **correct** word w_{t+1} , add it to context, move on

Training a transformer language model



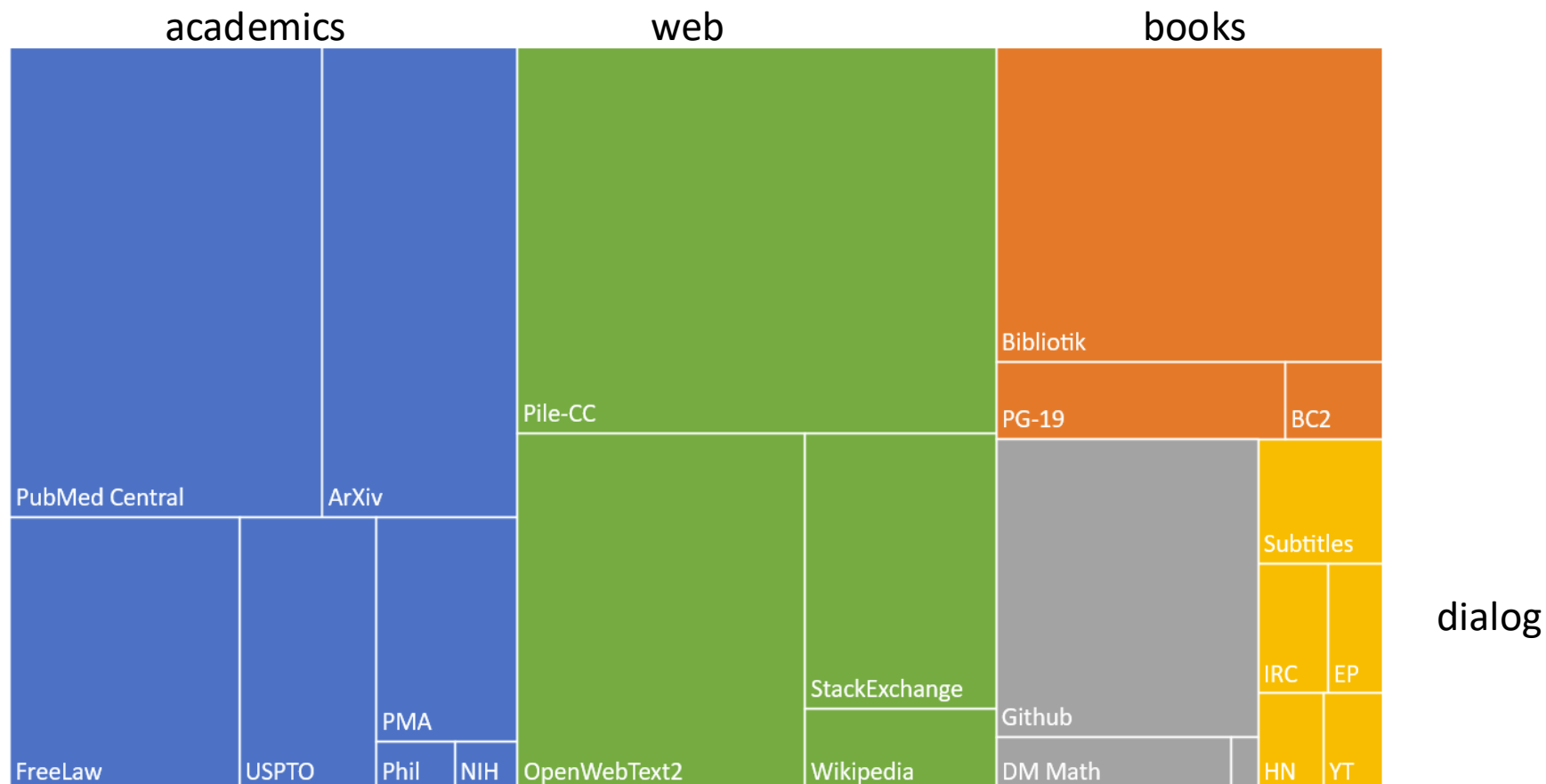
LLMs are mainly trained on the web

Common crawl, snapshots of the entire web produced by the non-profit Common Crawl with billions of pages

Colossal Clean Crawled Corpus (C4; [Raffel et al. 2020](#)), 156 billion tokens of English, filtered

What's in it? Mostly patent text documents, Wikipedia, and news sites

The Pile: a pretraining corpus



Filtering for quality and safety

Quality is subjective

- Many LLMs attempt to match Wikipedia, books, particular websites
- Need to remove boilerplate, adult content
- Deduplication at many levels (URLs, documents, even lines)

Safety also subjective

- Toxicity detection is important, although that has mixed results
- Can mistakenly flag data written in dialects like African American English

There are problems with scraping from the web



**Authors Sue OpenAI Claiming Mass Copyright
Infringement of Hundreds of Thousands of Novels**

The Times Sues OpenAI and Microsoft Over A.I. Use of Copyrighted Work

Millions of articles from The New York Times were used to train chatbots that now compete with it, the lawsuit said.



There are problems with scraping from the web

Copyright: much of the text in these datasets is copyrighted

- Not clear if fair use doctrine in US allows for this use
- This remains an open legal question across the world

Data consent

- Website owners can indicate they don't want their site crawled

Privacy:

- Websites can contain private IP addresses and phone numbers

Skew:

- Training data is disproportionately generated by authors from the US which probably skews resulting topics and opinions

Large Language Models

Pretraining Large Language Models

Large Language Models

Finetuning

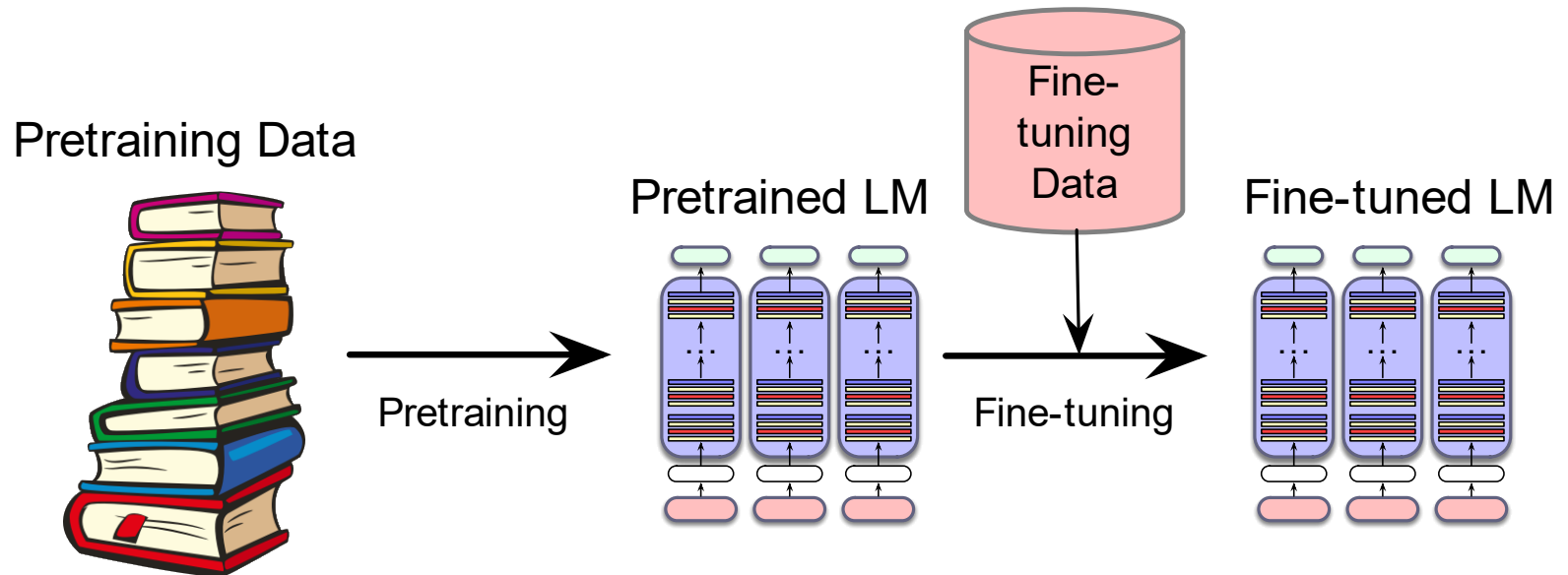
Finetuning for adaptation to new domains

What happens if we need our LLM to work well on a domain it didn't see in pretraining?

Perhaps some specific medical or legal domain?

Or maybe a multilingual LM needs to see more data on some language that was rare in pretraining?

Finetuning



"Finetuning" means 4 different things

We'll discuss 1 here, and 3 in later lectures

In all four cases, finetuning means:

**taking a pretrained model and further adapting
some or all of its parameters to some new data**

1. Finetuning as "continued pretraining" on new data

- Further train all the parameters of model on new data
 - using the same method (word prediction) and loss function (cross-entropy loss) as for pretraining.
 - as if the new data were at the tail end of the pretraining data
- Hence sometimes called **continued pretraining**

Large Language Models

Finetuning

Large
Language
Models

Evaluating Large Language
Models

Better LMs are better at predicting text

Reminder of the chain rule:

$$\begin{aligned} P(w_{1:n}) &= P(w_1)P(w_2|w_1)P(w_3|w_{1:2}) \dots P(w_n|w_{1:n-1}) \\ &= \prod_{i=1}^n P(w_i|w_{<i}) \end{aligned}$$

So given a text $w_{1:n}$ we could just compare the log likelihood from two LMs:

$$\text{log likelihood}(w_{1:n}) = \log \prod_{i=1}^n P(w_i|w_{<i})$$

But raw log-likelihood has problems

Probability depends on size of test set

- Probability gets smaller the longer the text
- We would prefer a metric that is **per-word**, normalized by length

Perplexity is normalized for length

Perplexity is the inverse probability of the test set, normalized by the number of words

(The inverse comes from the original definition of perplexity from cross-entropy rate in information theory)

Probability range is $[0,1]$, perplexity range is $[1,\infty]$

Perplexity

So just as for n-gram grammars, we use perplexity to measure how well the LM predicts unseen text

The perplexity of a model θ on an unseen test set is the **inverse probability that θ assigns to the test set, normalized by the test set length.**

For a test set of n tokens $w_{1:n}$ the perplexity is :

$$\begin{aligned}\text{Perplexity}_{\theta}(w_{1:n}) &= P_{\theta}(w_{1:n})^{-\frac{1}{n}} \\ &= \sqrt[n]{\frac{1}{P_{\theta}(w_{1:n})}} = \sqrt[n]{\prod_{i=1}^n \frac{1}{P_{\theta}(w_i | w_{<i})}}\end{aligned}$$

Perplexity

- The higher the probability of the word sequence, the lower the perplexity.
- Thus the lower the perplexity of a model on the data, the better the model.
- **Minimizing perplexity is the same as maximizing probability**

Also: perplexity is sensitive to length/tokenization so best used when comparing LMs that use the same tokenizer.

Many other factors that we evaluate, like:

Size

Big models take lots of GPUs and time to train, memory to store

Energy usage

Can measure kWh or kilograms of CO2 emitted

Fairness

Benchmarks measure gendered and racial stereotypes, or decreased performance for language from or about some groups.

Large
Language
Models

Ethical and Safety Issues in Large Language Models

Hallucination

*Chatbots May 'Hallucinate'
More Often Than Many Realize*

*What Can You Do When A.I. Lies
About You?*

People have little protection or recourse when the technology creates and spreads falsehoods about them.

**Air Canada loses court case after its chatbot hallucinated
fake policies to a customer**

The airline argued that the chatbot itself was liable. The court disagreed.

Privacy

**How Strangers Got My Email
Address From ChatGPT's Model**

Abuse and Toxicity

The New AI-Powered Bing Is Threatening Users.

Cleaning Up ChatGPT Takes Heavy Toll on Human Workers

Contractors in Kenya say they were traumatized by effort to screen out descriptions of violence and sexual abuse during run-up to OpenAI's hit chatbot

Lots more

Harm (suggesting dangerous actions)

Fraud

Emotional dependence

Bias

Mary Shelley's Frankenstein

Centered on the problem of creating artificial agents without considering ethical and humanistic concerns.



Large
Language
Models

Ethical and Safety Issues in Large Language Models

Session 10

Unit III

ransformers and Large Language Models

Topic 3.2

The transformer model

Transformers

Introduction to Transformers

LLMs are built out of transformers

Transformer: a specific kind of network architecture, like a fancier feedforward network, but based on attention

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin* †
illia.polosukhin@gmail.com

A very approximate timeline

1990 Static Word Embeddings

2003 Neural Language Model

2008 Multi-Task Learning

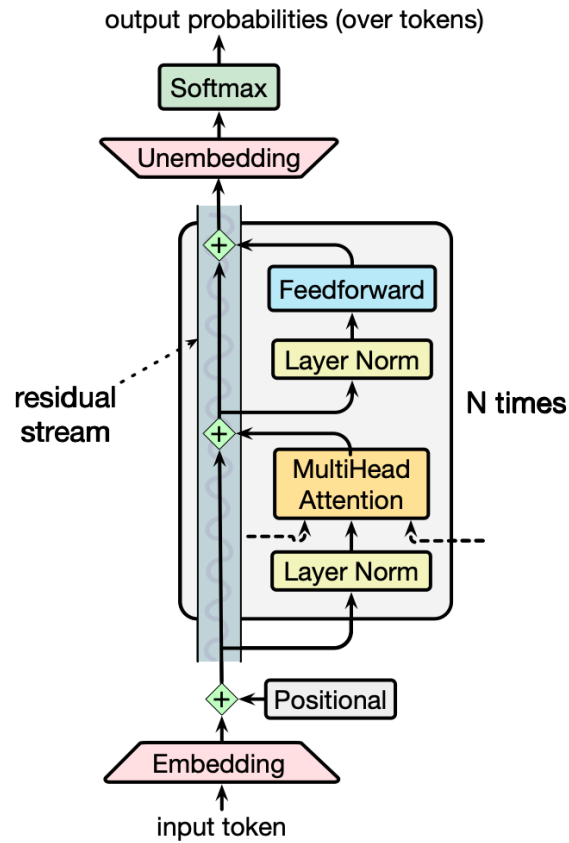
2015 Attention

2017 Transformer

2018 Contextual Word Embeddings and Pretraining

2019 Prompting

The transformer

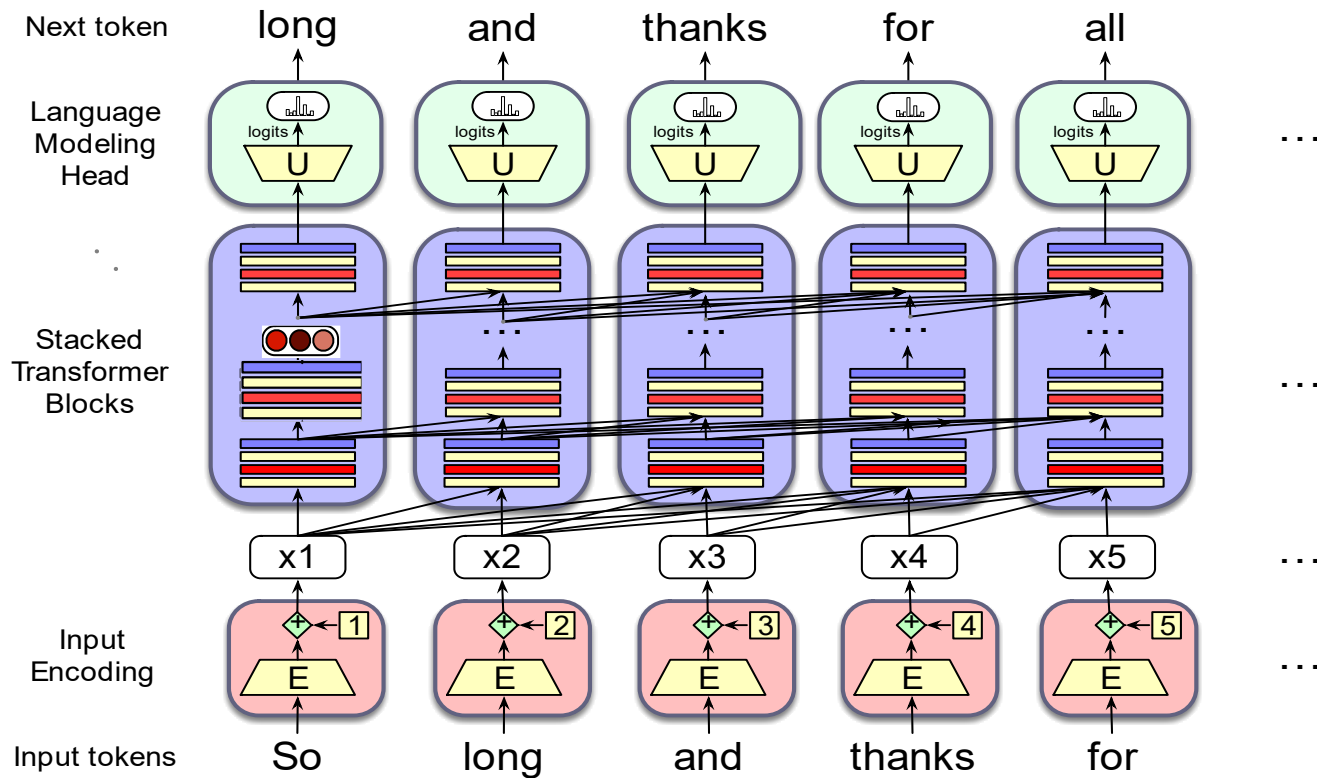


Transformers

Attention

Instead of starting with the big picture

Let's consider the embeddings for an individual word from a particular layer



Problem with static embeddings (word2vec)

They are static! The embedding for a word doesn't reflect how its meaning changes in context.

The chicken didn't cross the road because  it was too tired

What is the meaning represented in the static embedding for "it"?

Contextual Embeddings

- Intuition: a representation of meaning of a word should be different in different contexts!
- **Contextual Embedding**: each word has a different vector that expresses different meanings depending on the surrounding words
- How to compute contextual embeddings?
 - **Attention**

Contextual Embeddings

The chicken didn't cross the road because it

What should be the properties of "it"?

The chicken didn't cross the road because it was too **tired**

The chicken didn't cross the road because it was too **wide**

At this point in the sentence, it's probably referring to either the chicken or the street

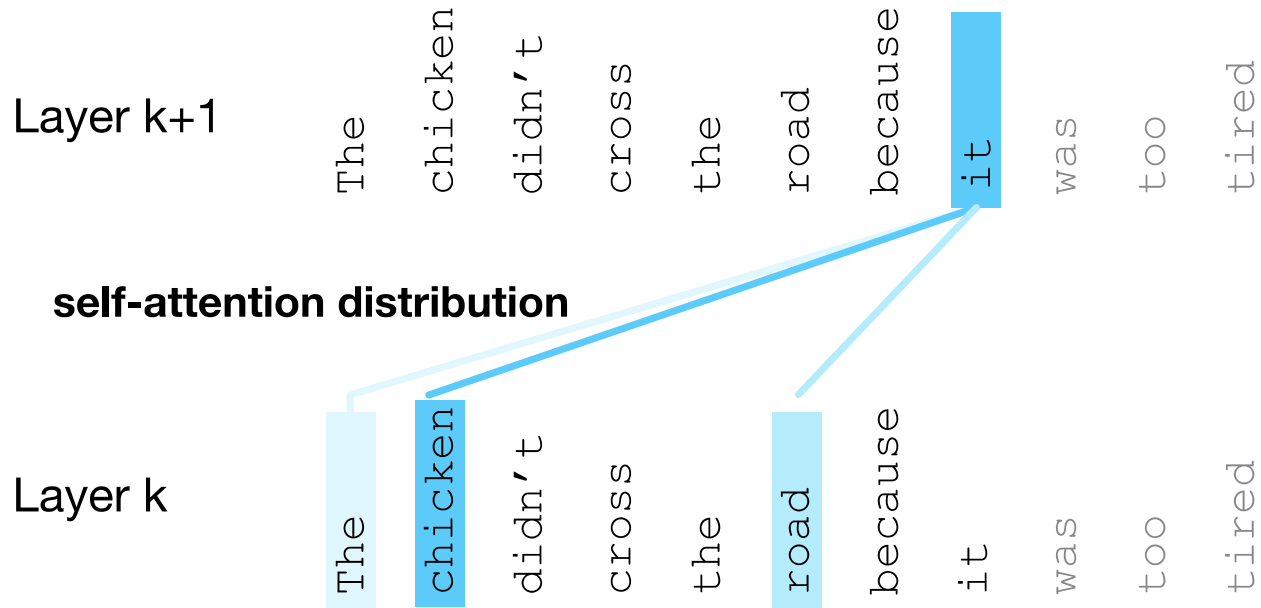
Intuition of attention

Build up the contextual embedding from a word by selectively integrating information from all the neighboring words

We say that a word "attends to" some neighboring words more than others

Intuition of attention:

columns corresponding to input tokens

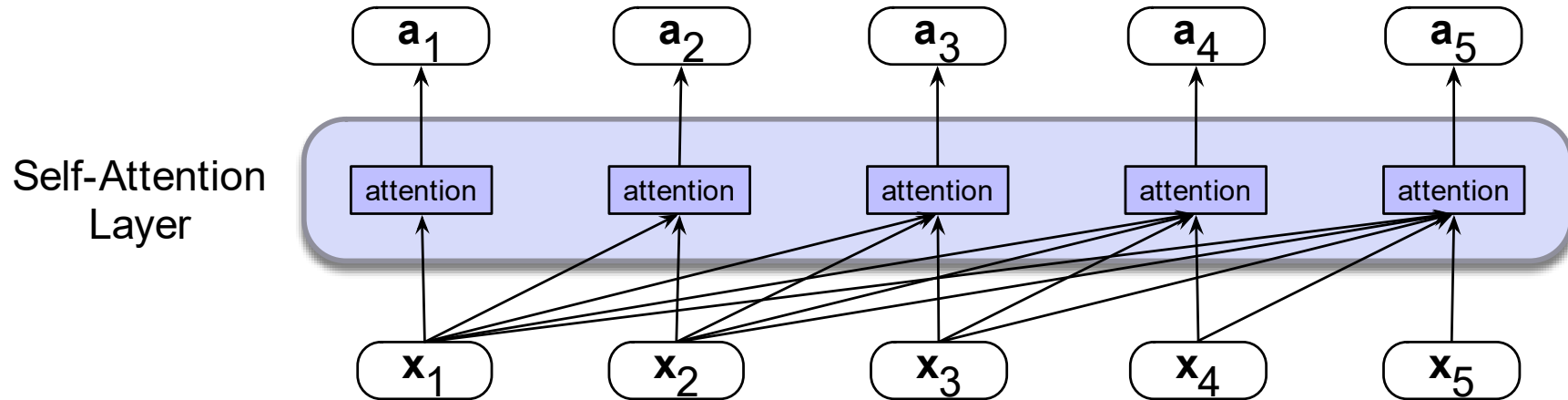


Attention definition

A mechanism for helping compute the embedding for a token by selectively attending to and integrating information from surrounding tokens (at the previous layer).

More formally: a method for doing a weighted sum of vectors.

Attention is left-to-right



Simplified version of attention: a sum of prior words weighted by their similarity with the current

Given a sequence of token^{word} embeddings:

$\mathbf{x}_1 \quad \mathbf{x}_2 \quad \mathbf{x}_3 \quad \mathbf{x}_4 \quad \mathbf{x}_5 \quad \mathbf{x}_6 \quad \mathbf{x}_7 \quad \mathbf{x}_i$

Produce: $\mathbf{a}_i$ = a weighted sum of $\mathbf{x}_1$ through $\mathbf{x}_7$ (and $\mathbf{x}_i$)
Weighted by their similarity to $\mathbf{x}_i$

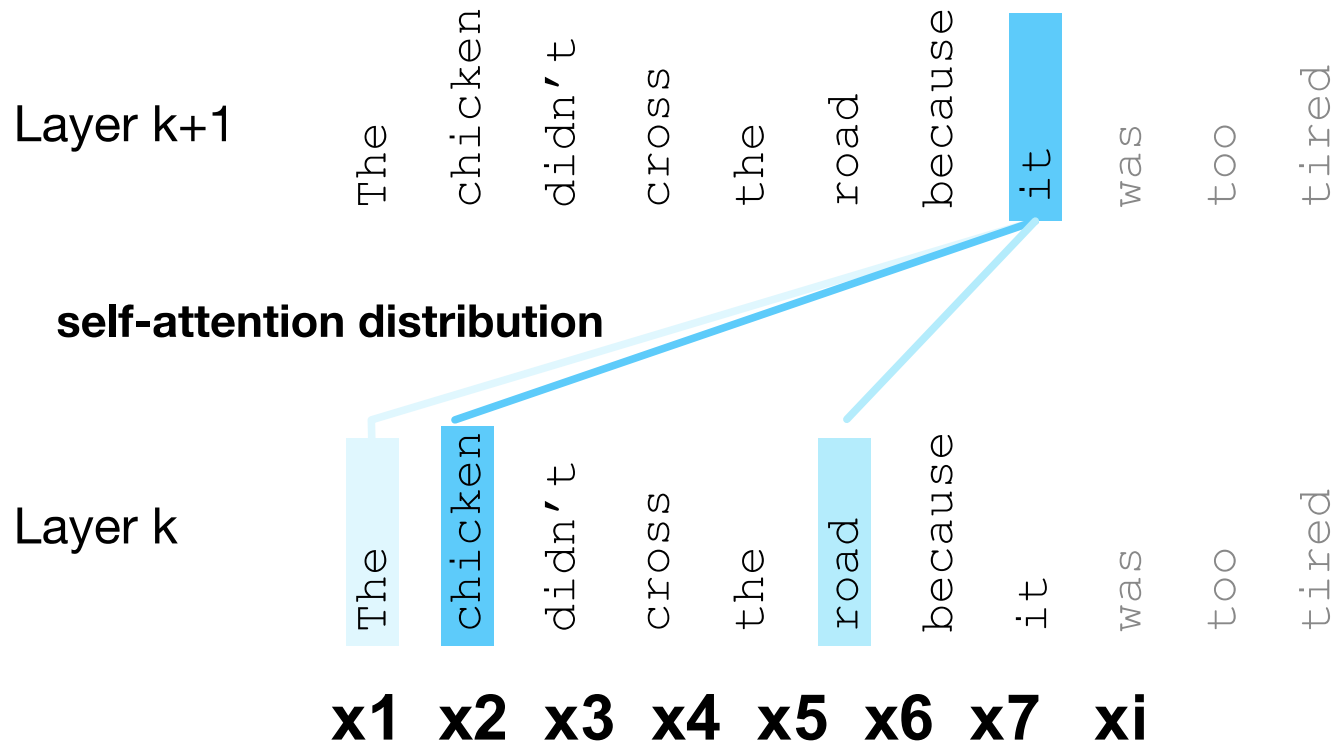
$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j$$

Intuition of attention:

columns corresponding to input tokens

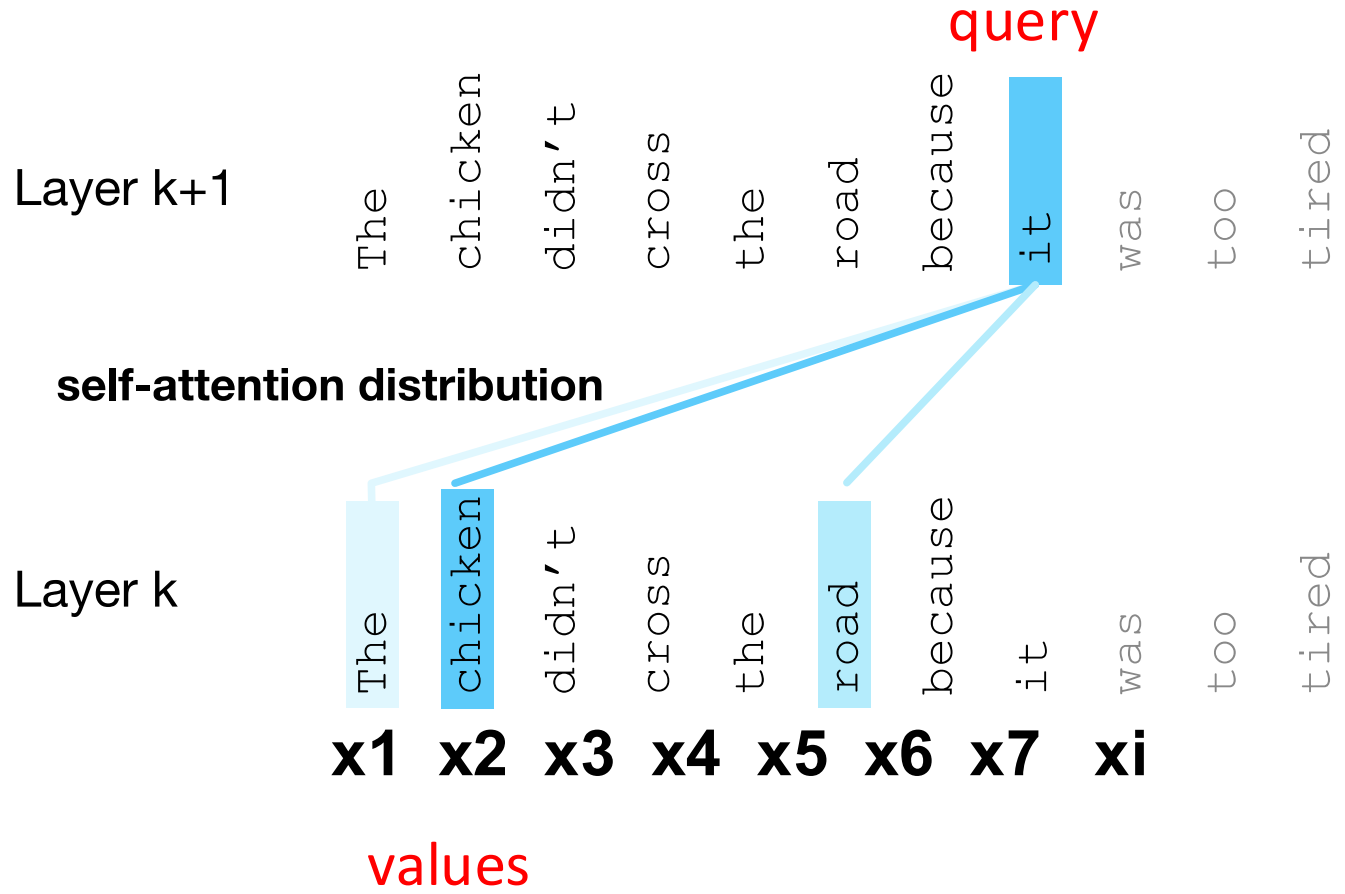


An Actual Attention Head: slightly more complicated

High-level idea: instead of using vectors (like x_i and x_4) directly, we'll represent 3 separate roles each vector x_i plays:

- **query**: *As the current element* being compared to the preceding inputs.
- **key**: *as a preceding input* that is being compared to the current element to determine a similarity
- **value**: a value of a preceding element that gets weighted and summed

Attention intuition



An Actual Attention Head: slightly more complicated

We'll use matrices to project each vector $\mathbf{x}_i$ into a representation of its role as query, key, value:

- **query: $\mathbf{W}^Q$**
- **key: $\mathbf{W}^K$**
- **value: $\mathbf{W}^V$**

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

An Actual Attention Head: slightly more complicated

Given these 3 representation of $\mathbf{x}_i$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

To compute similarity of current element $\mathbf{x}_i$ with some prior element $\mathbf{x}_j$

We'll use dot product between $\mathbf{q}_i$ and $\mathbf{k}_j$.

And instead of summing up $\mathbf{x}_j$, we'll sum up $\mathbf{v}_j$

Final equations for one attention head

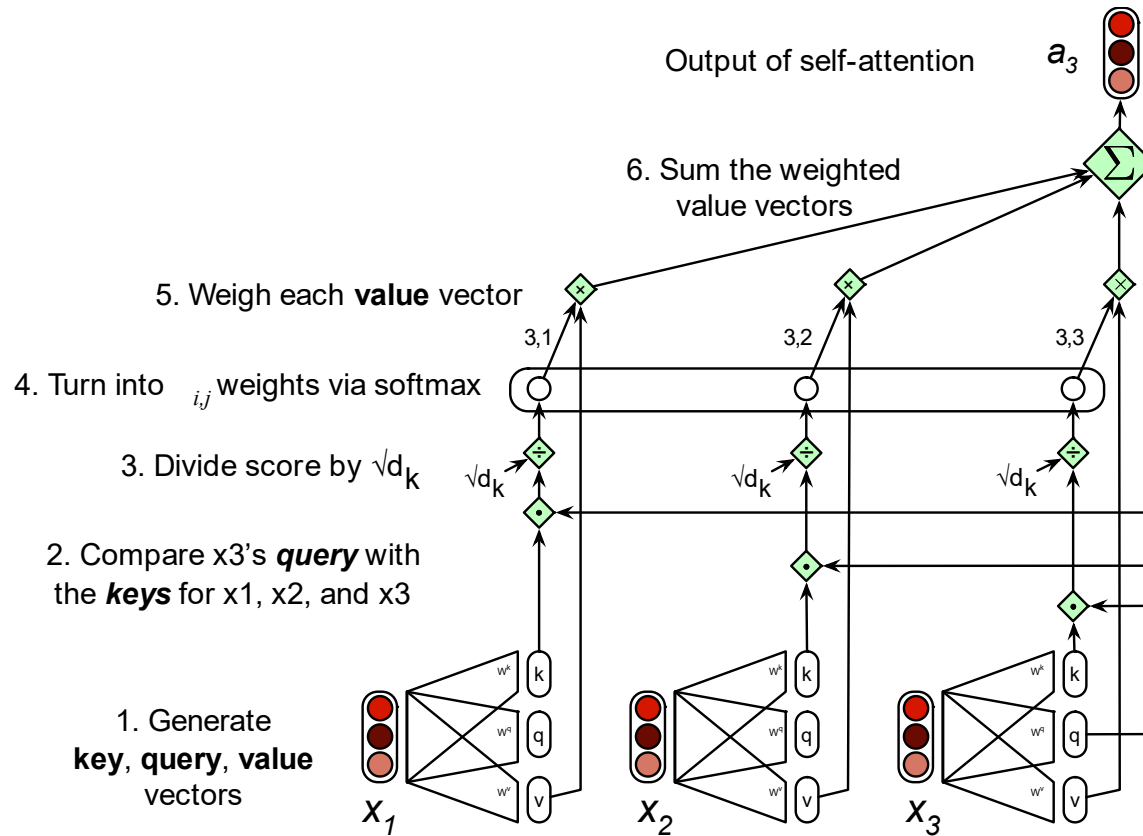
$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_j = \mathbf{x}_j \mathbf{W}^K; \quad \mathbf{v}_j = \mathbf{x}_j \mathbf{W}^V$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j$$

Calculating the value of a_3



Actual Attention: slightly more complicated

- Instead of one attention head, we'll have lots of them!
- Intuition: each head might be attending to the context for different purposes
 - Different linguistic relationships or patterns in the context

$$\mathbf{q}_i^c = \mathbf{x}_i \mathbf{W}^{\mathbf{Q}^c}; \quad \mathbf{k}_j^c = \mathbf{x}_j \mathbf{W}^{\mathbf{K}^c}; \quad \mathbf{v}_j^c = \mathbf{x}_j \mathbf{W}^{\mathbf{V}^c}; \quad \forall c \quad 1 \leq c \leq h$$

$$\text{score}^c(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i^c \cdot \mathbf{k}_j^c}{\sqrt{d_k}}$$

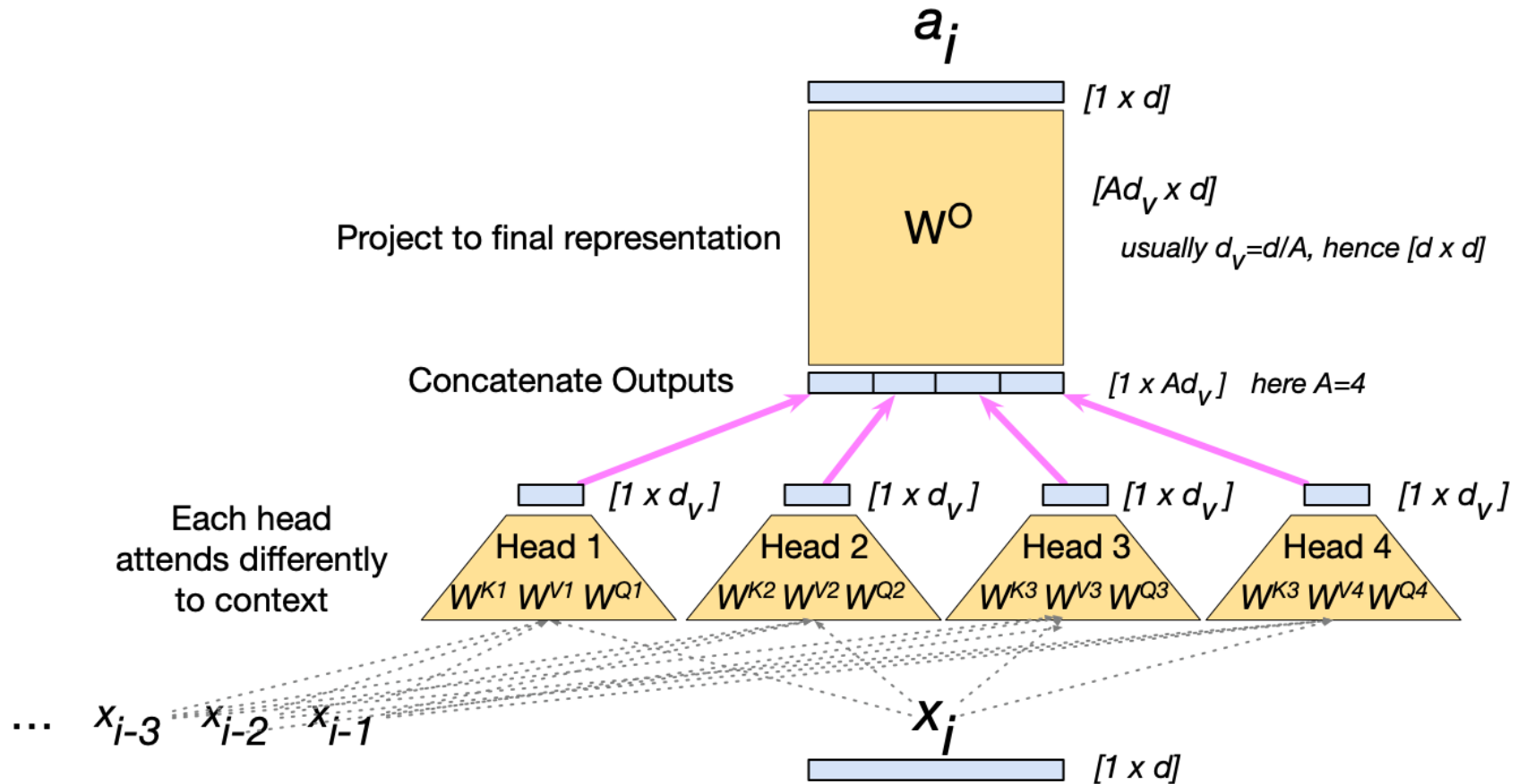
$$\alpha_{ij}^c = \text{softmax}(\text{score}^c(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

$$\text{head}_i^c = \sum_{j \leq i} \alpha_{ij}^c \mathbf{v}_j^c$$

$$\mathbf{a}_i = (\text{head}^1 \oplus \text{head}^2 \dots \oplus \text{head}^h) \mathbf{W}^O$$

$$\text{MultiHeadAttention}(\mathbf{x}_i, [\mathbf{x}_1, \dots, \mathbf{x}_N]) = \mathbf{a}_i$$

Multi-head attention



Summary

Attention is a method for enriching the representation of a token by incorporating contextual information

The result: the embedding for each word will be different in different contexts!

Contextual embeddings: a representation of word meaning in its context.

We'll see in the next lecture that attention can also be viewed as a way to move information from one token to another.

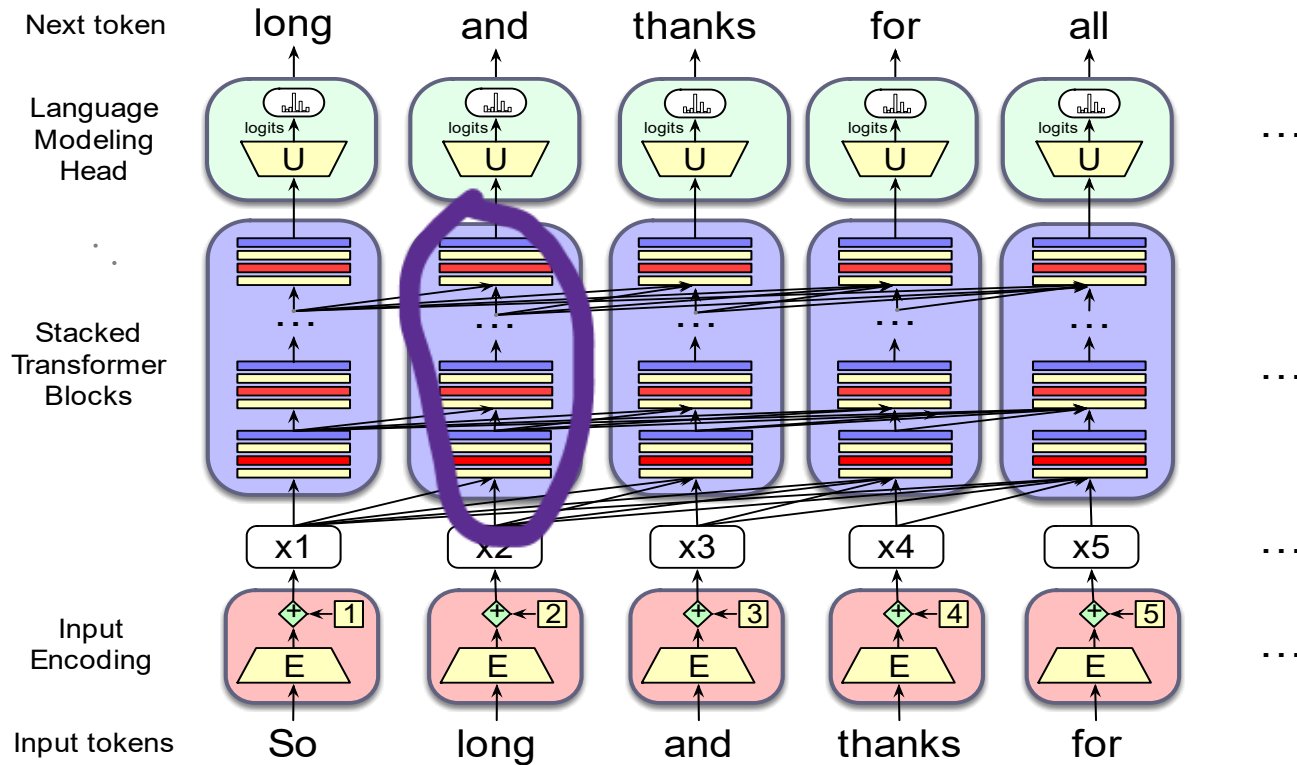
Transformers

Attention

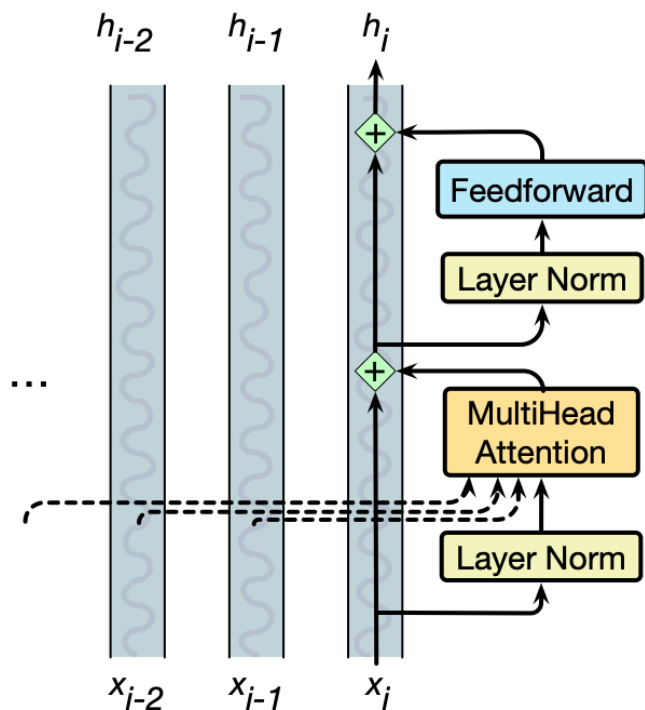
Transformers

The Transformer Block

Reminder: transformer language model

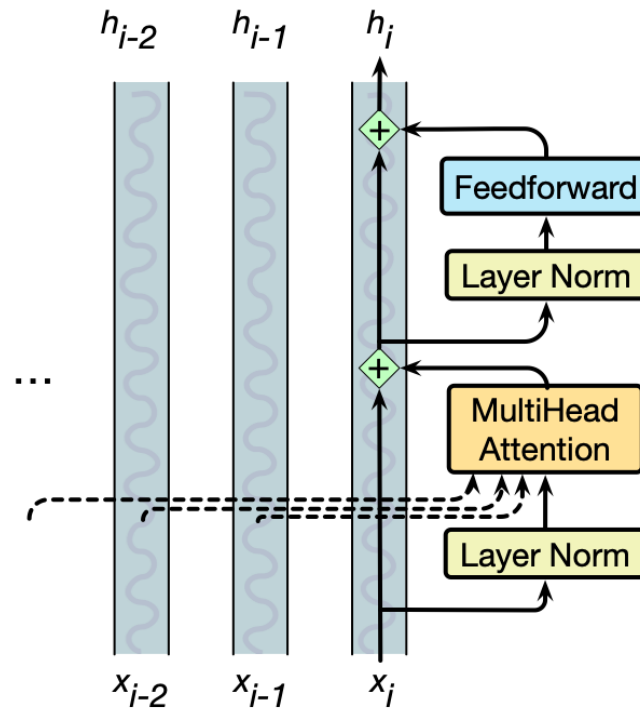


The residual stream: each token gets passed up and modified

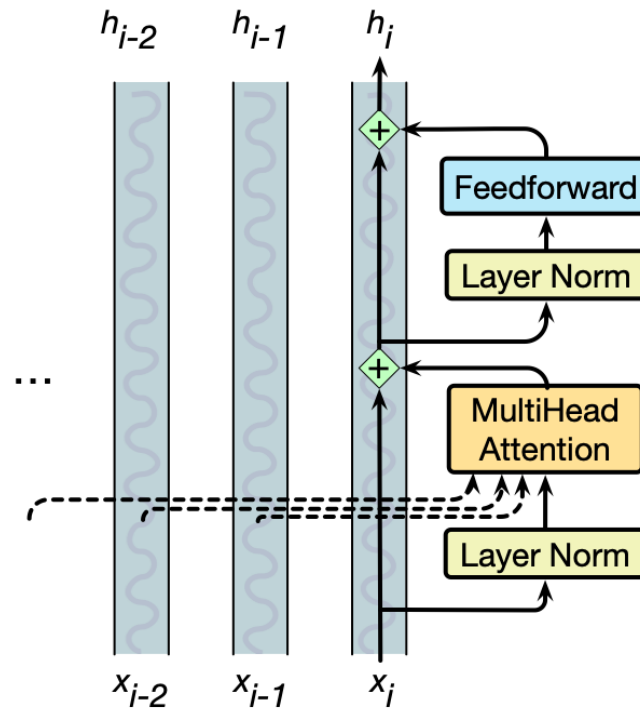


We'll need nonlinearities, so a feedforward layer

$$\text{FFN}(\mathbf{x}_i) = \text{ReLU}(\mathbf{x}_i \mathbf{W}_1 + b_1) \mathbf{W}_2 + b_2$$



Layer norm: the vector $\mathbf{x}_i$ is normalized twice



Layer Norm

Layer norm is a variation of the z-score from statistics, applied to a single vector in a hidden layer

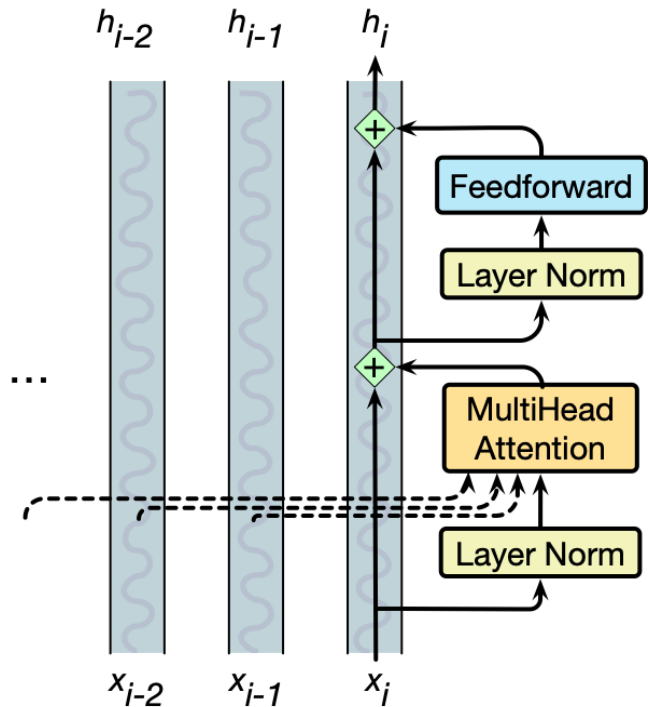
$$\mu = \frac{1}{d} \sum_{i=1}^d x_i$$

$$\sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2}$$

$$\hat{\mathbf{x}} = \frac{(\mathbf{x} - \mu)}{\sigma}$$

$$\text{LayerNorm}(\mathbf{x}) = \gamma \frac{(\mathbf{x} - \mu)}{\sigma} + \beta$$

Putting together a single transformer block



$$\mathbf{t}_i^1 = \text{LayerNorm}(\mathbf{x}_i)$$

$$\mathbf{t}_i^2 = \text{MultiHeadAttention}(\mathbf{t}_i^1, [\mathbf{x}_1^1, \dots, \mathbf{x}_N^1])$$

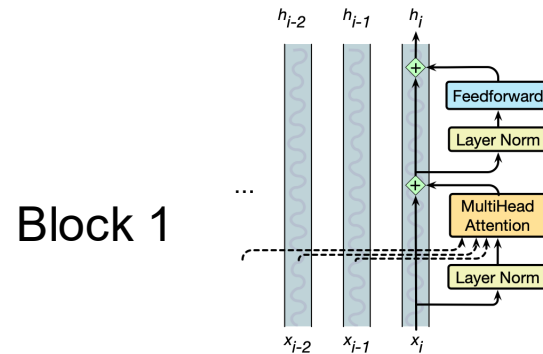
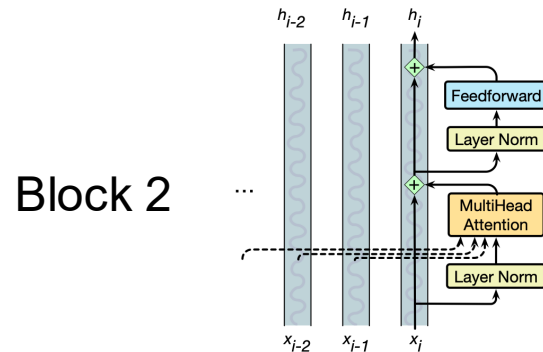
$$\mathbf{t}_i^3 = \mathbf{t}_i^2 + \mathbf{x}_i$$

$$\mathbf{t}_i^4 = \text{LayerNorm}(\mathbf{t}_i^3)$$

$$\mathbf{t}_i^5 = \text{FFN}(\mathbf{t}_i^4)$$

$$\mathbf{h}_i = \mathbf{t}_i^5 + \mathbf{t}_i^3$$

A transformer is a stack of these blocks
so all the vectors are of the same dimensionality d

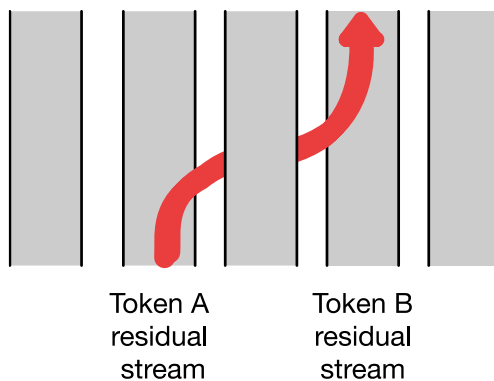


Residual streams and attention

Notice that all parts of the transformer block apply to 1 residual stream (1 token).

Except attention, which takes information from other tokens

[Elhage et al. \(2021\)](#) show that we can view attention heads as literally moving information from the residual stream of a neighboring token into the current stream.



Transformers

The Transformer Block

Transformers

Parallelizing Attention Computation

Parallelizing computation using **X**

For attention/transformer block we've been computing a **single** output at a **single** time step i in a **single** residual stream.

But we can pack the N tokens of the input sequence into a single matrix **X** of size $[N \times d]$.

Each row of **X** is the embedding of one token of the input.

X can have 1K-32K rows, each of the dimensionality of the embedding d (the **model dimension**)

$$\mathbf{Q} = \mathbf{XW}^{\mathbf{Q}}; \mathbf{K} = \mathbf{XW}^{\mathbf{K}}; \mathbf{V} = \mathbf{XW}^{\mathbf{V}}$$

$$QK^T$$

Now can do a single matrix multiply to combine Q and K^T

N	q1•k1	q1•k2	q1•k3	q1•k4
	q2•k1	q2•k2	q2•k3	q2•k4
	q3•k1	q3•k2	q3•k3	q3•k4
	q4•k1	q4•k2	q4•k3	q4•k4
N				

Parallelizing attention

- Scale the scores, take the softmax, and then multiply the result by V resulting in a matrix of shape $N \times d$
 - An attention vector for each input token

$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \right) \right) \mathbf{V}$$

Masking out the future

$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}} \right) \right) \mathbf{V}$$

- What is this mask function?
 $\mathbf{QK}^\top$ has a score for each query dot every key, *including those that follow the query.*
- Guessing the next word is pretty simple if you already know it!

Masking out the future

$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{QK}^T}{\sqrt{d_k}} \right) \right) \mathbf{V}$$

Add $-\infty$ to cells in upper triangle
The softmax will turn it to 0

N

q1•k1	$-\infty$	$-\infty$	$-\infty$
q2•k1	q2•k2	$-\infty$	$-\infty$
q3•k1	q3•k2	q3•k3	$-\infty$
q4•k1	q4•k2	q4•k3	q4•k4

N

Another point: Attention is quadratic in length

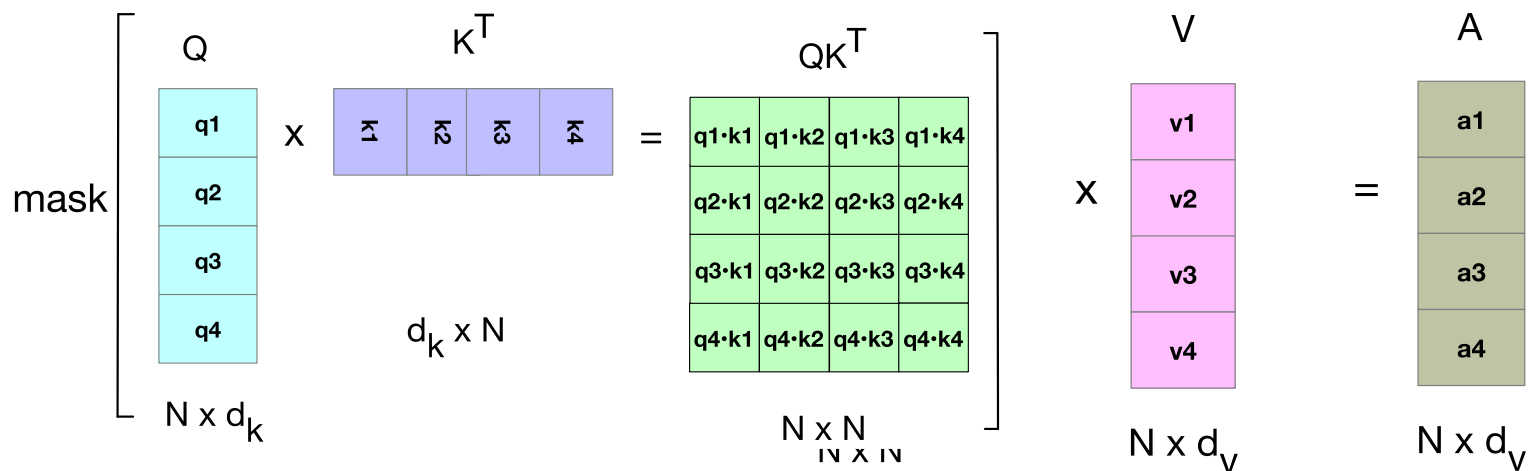
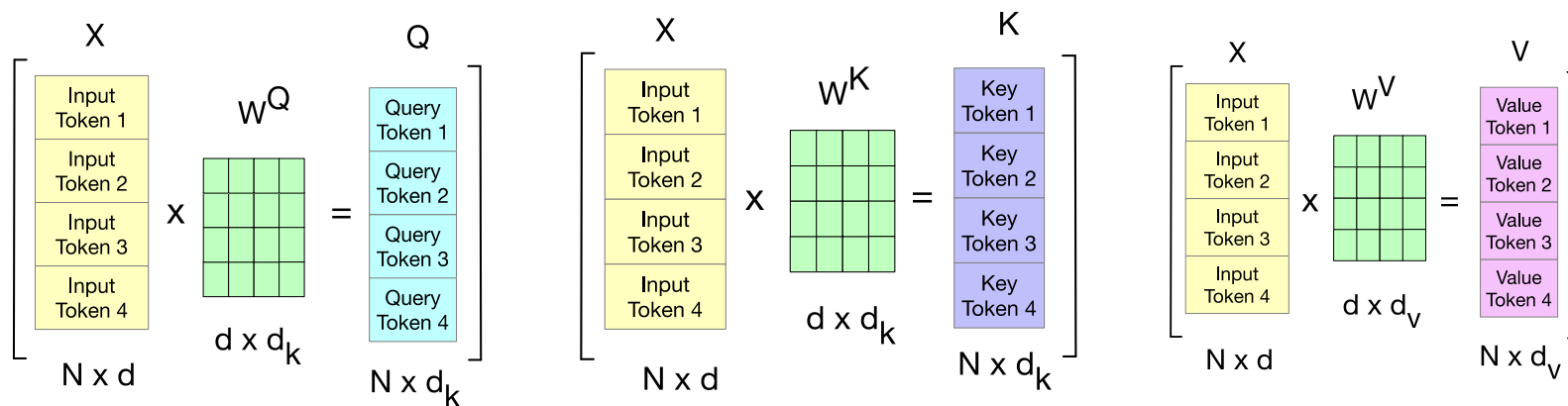
$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{QK}^T}{\sqrt{d_k}} \right) \right) \mathbf{V}$$

N

q1•k1	−∞	−∞	−∞
q2•k1	q2•k2	−∞	−∞
q3•k1	q3•k2	q3•k3	−∞
q4•k1	q4•k2	q4•k3	q4•k4

N

Attention again



Parallelizing Multi-head Attention

$$\mathbf{Q}^i = \mathbf{XW}^{\mathbf{Q}^i}; \quad \mathbf{K}^i = \mathbf{XW}^{\mathbf{K}^i}; \quad \mathbf{V}^i = \mathbf{XW}^{\mathbf{V}^i}$$

$$\mathbf{head}_i = \text{SelfAttention}(\mathbf{Q}^i, \mathbf{K}^i, \mathbf{V}^i) = \text{softmax}\left(\frac{\mathbf{Q}^i \mathbf{K}^{i\top}}{\sqrt{d_k}}\right) \mathbf{V}^i$$

$$\text{MultiHeadAttention}(\mathbf{X}) = (\mathbf{head}_1 \oplus \mathbf{head}_2 \dots \oplus \mathbf{head}_h) \mathbf{W}^O$$

Parallelizing Multi-head Attention

$$\mathbf{O} = \text{LayerNorm}(\mathbf{X} + \text{MultiHeadAttention}(\mathbf{X}))$$

$$\mathbf{H} = \text{LayerNorm}(\mathbf{O} + \text{FFN}(\mathbf{O}))$$

or

$$\mathbf{T}^1 = \text{MultiHeadAttention}(\mathbf{X})$$

$$\mathbf{T}^2 = \mathbf{X} + \mathbf{T}^1$$

$$\mathbf{T}^3 = \text{LayerNorm}(\mathbf{T}^2)$$

$$\mathbf{T}^4 = \text{FFN}(\mathbf{T}^3)$$

$$\mathbf{T}^5 = \mathbf{T}^4 + \mathbf{T}^3$$

$$\mathbf{H} = \text{LayerNorm}(\mathbf{T}^5)$$

Transformers

Parallelizing Attention Computation

Transformers

Input and output: Position
embeddings and the
Language Model Head

Token and Position Embeddings

The matrix X (of shape $[N \times d]$) has an embedding for each word in the context.

This embedding is created by adding two distinct embeddings for each input

- token embedding
- positional embedding

Token Embeddings

Embedding matrix E has shape $[|V| \times d]$.

- One row for each of the $|V|$ tokens in the vocabulary.
- Each word is a row vector of d dimensions

Given: string "*Thanks for all the*"

1. Tokenize with BPE and convert into vocab indices

$w = [5, 4000, 10532, 2224]$

2. Select the corresponding rows from E , each row an embedding

- (row 5, row 4000, row 10532, row 2224).

Position Embeddings

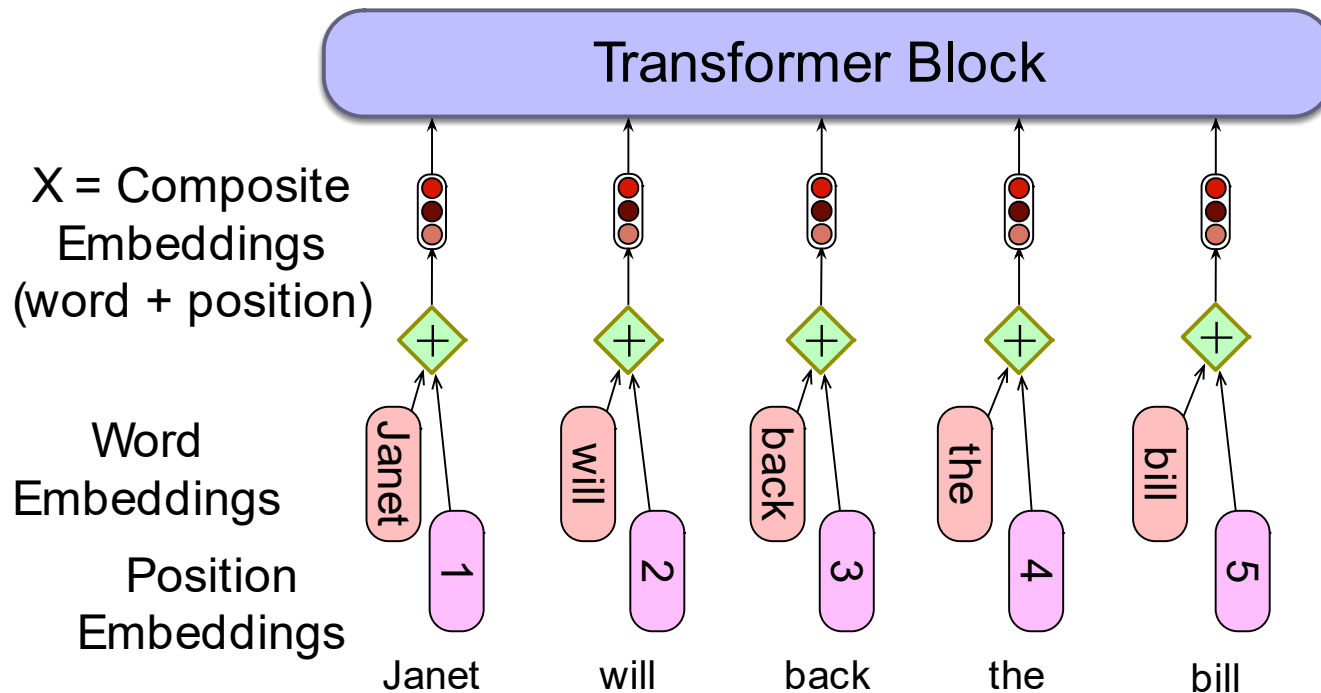
There are many methods, but we'll just describe the simplest: absolute position.

Goal: learn a position embedding matrix E_{pos} of shape $[1 \times N]$.

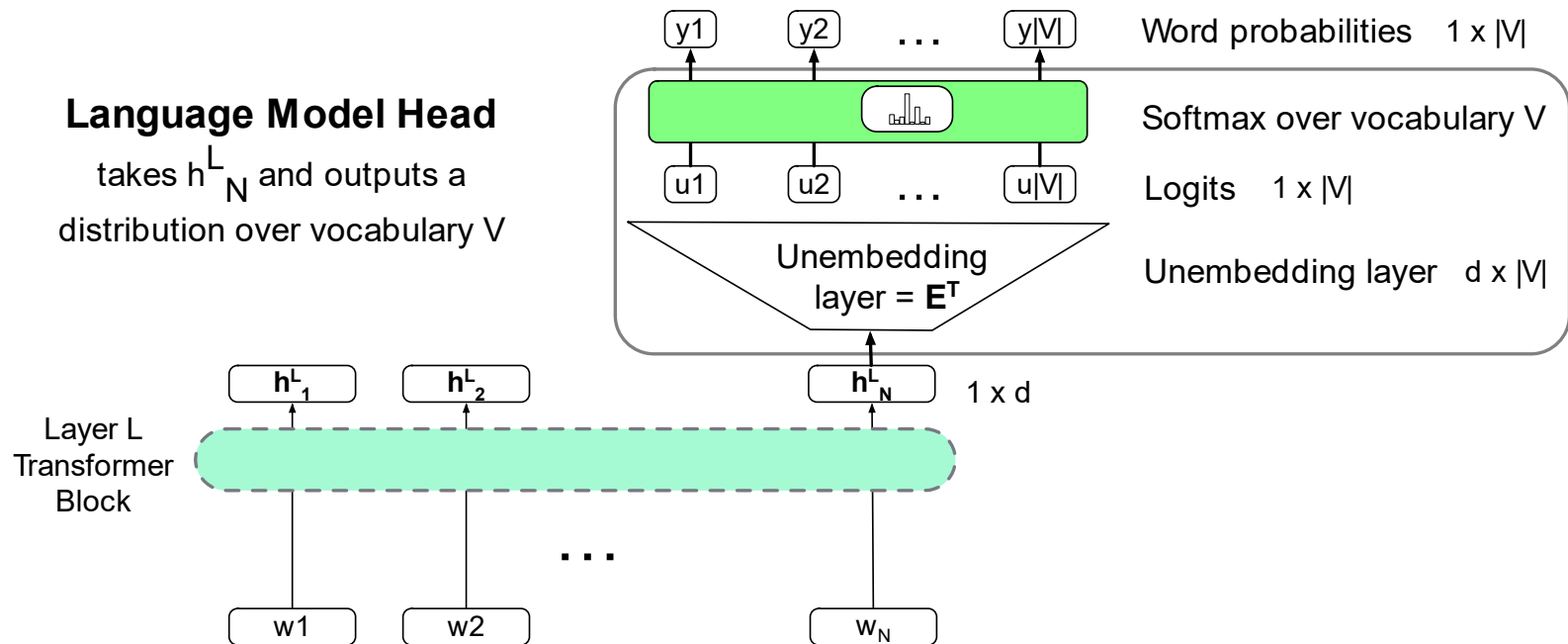
Start with randomly initialized embeddings

- one for each integer up to some maximum length.
- i.e., just as we have an embedding for token *fish*, we'll have an embedding for position 3 and position 17.
- As with word embeddings, these position embeddings are learned along with other parameters during training.

Each $\mathbf{x}$ is just the sum of word and position embeddings

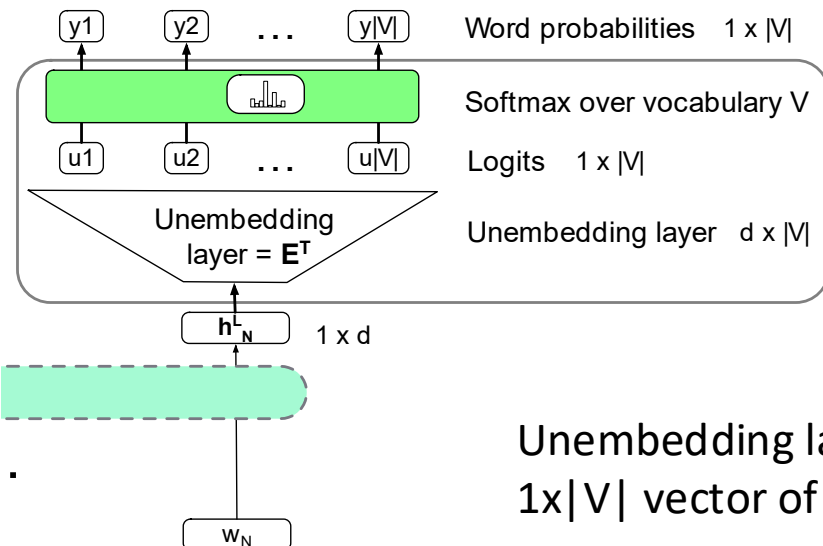


Language modeling head



Language modeling head

Unembedding layer: linear layer projects from h_N^L (shape $[1 \times d]$) to logit vector



Why "unembedding"? **Tied** to E^T

Weight tying, we use the same weights for two different matrices

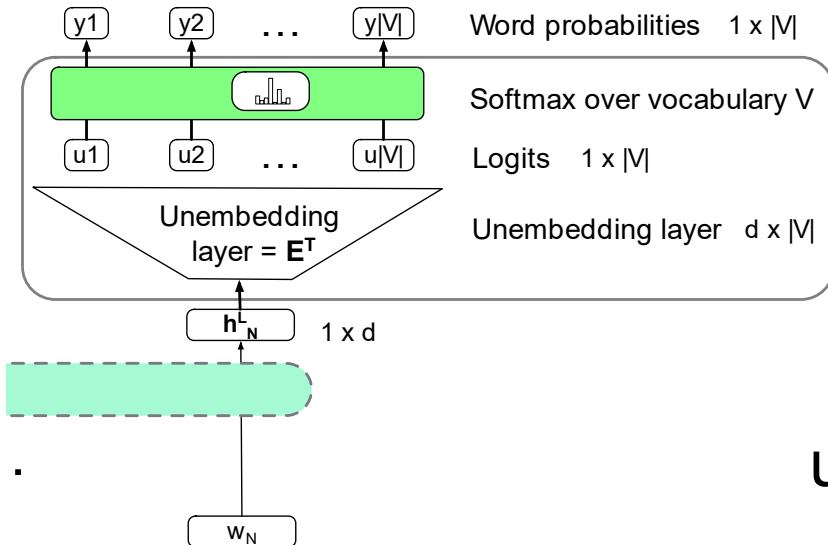
Unembedding layer maps from an embedding to a $1 \times |V|$ vector of logits

Language modeling head

Logits, the score vector u

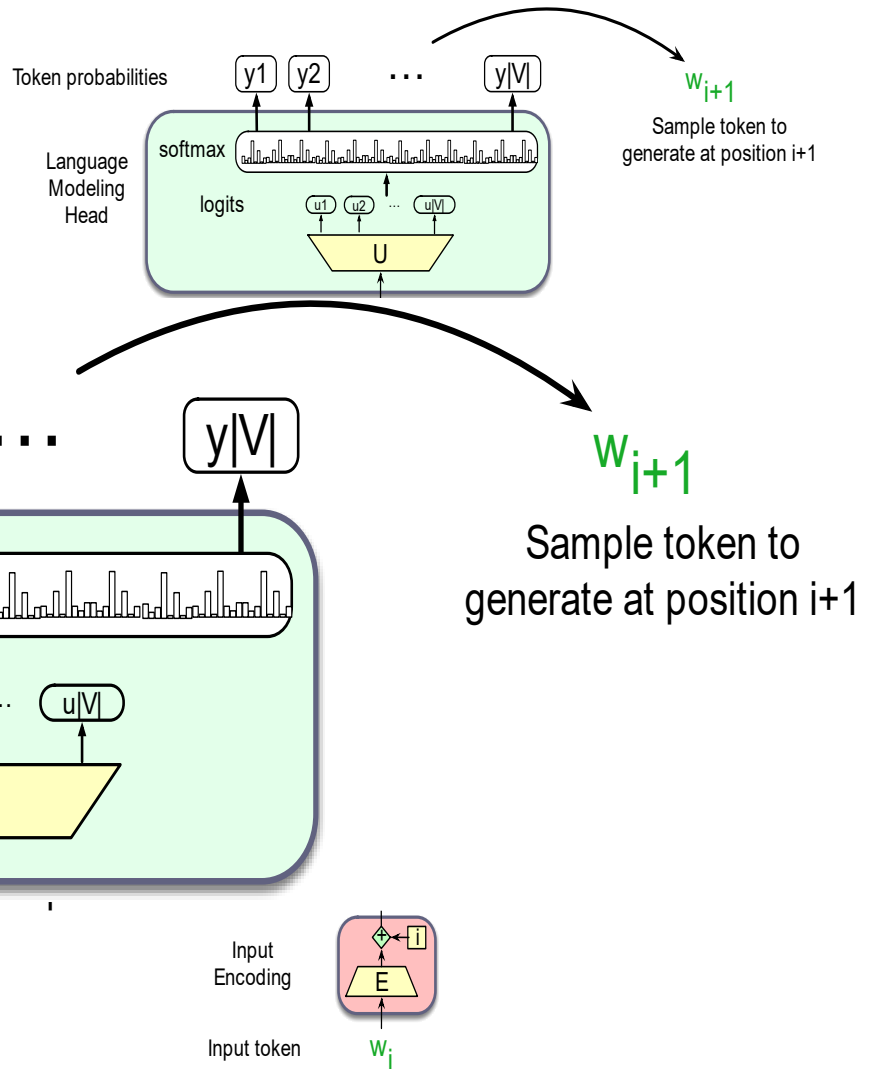
One score for each of the $|V|$ possible words in the vocabulary V .
Shape $1 \times |V|$.

Softmax turns the logits into probabilities over vocabulary.
Shape $1 \times |V|$.



$$u = h_N^L E^T$$
$$y = \text{softmax}(u)$$

The final
 n



Transformers

Input and output: Position
embeddings and the
Language Model Head

Large Language Models

Dealing with Scale

Scaling Laws

LLM performance depends on

- Model size: the number of parameters not counting embeddings
- Dataset size: the amount of training data
- Compute: Amount of compute (in FLOPS or etc

Can improve a model by adding parameters (more layers, wider contexts), more data, or training for more iterations

The performance of a large language model (the loss) scales as a power-law with each of these three

Scaling Laws

Loss L as a function of # parameters N , dataset size D , compute budget C (if other two are held constant)

$$L(N) = \left(\frac{N_c}{N} \right)^{\alpha_N}$$

$$L(D) = \left(\frac{D_c}{D} \right)^{\alpha_D}$$

$$L(C) = \left(\frac{C_c}{C} \right)^{\alpha_C}$$

Scaling laws can be used early in training to predict what the loss would be if we were to add more data or increase model size.

Number of non-embedding parameters N

$$\begin{aligned} N &\approx 2 d n_{\text{layer}} (2 d_{\text{attn}} + d_{\text{ff}}) \\ &\approx 12 n_{\text{layer}} d^2 \\ &\quad (\text{assuming } d_{\text{attn}} = d_{\text{ff}}/4 = d) \end{aligned}$$

Thus GPT-3, with $n = 96$ layers and dimensionality $d = 12288$, has $12 \times 96 \times 12288^2 \approx 175$ billion parameters.

KV Cache

In training, we can compute attention very efficiently in parallel:

$$\mathbf{A} = \text{softmax} \left(\frac{\mathbf{QK}^T}{\sqrt{d_k}} \right) \mathbf{V}$$

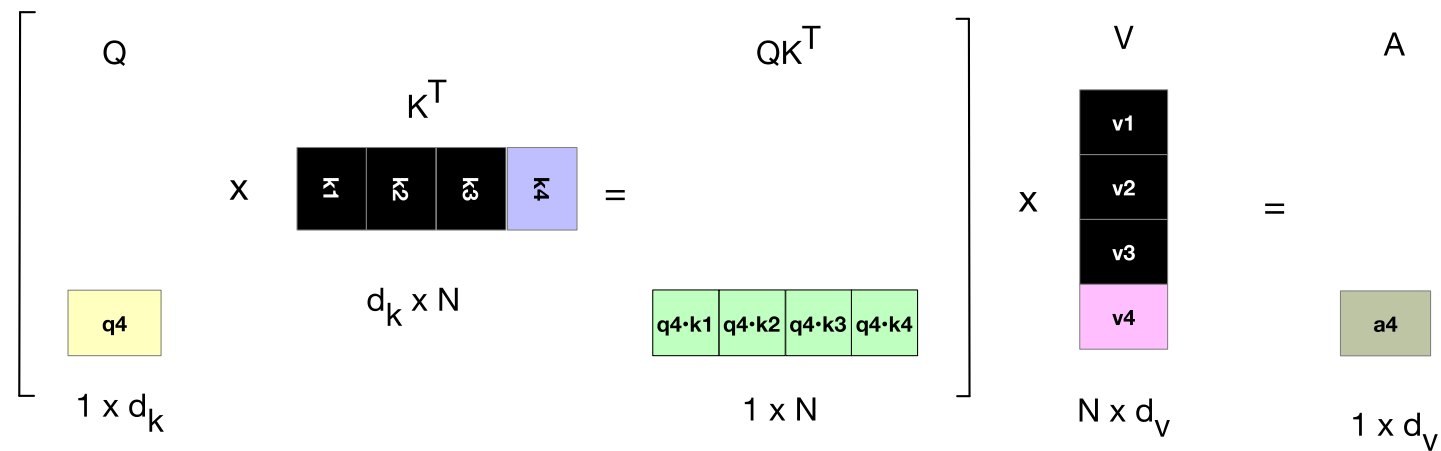
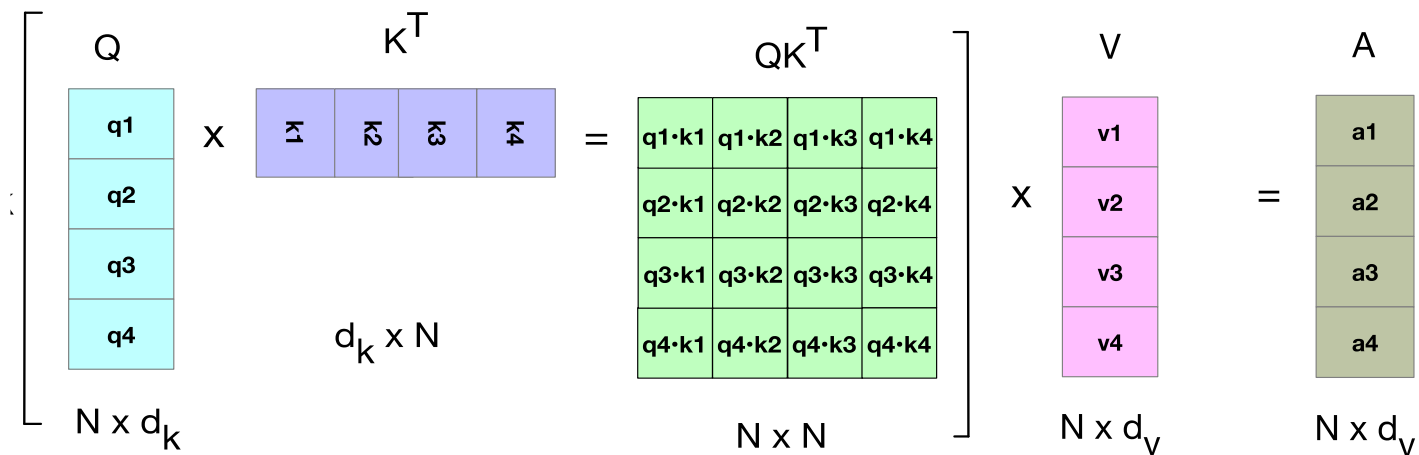
But not at inference! We generate the next tokens **one at a time!**

For a new token x , need to multiply by W^Q , W^K , and W^V to get query, key, values

But don't want to **recompute** the key and value vectors for all the prior tokens $x_{<i}$

Instead, store key and value vectors in memory in the KV cache, and then we can just grab them from the cache

KV Cache



Parameter-Efficient Finetuning

Adapting to a new domain by continued pretraining (finetuning) is a problem with huge LLMs.

- Enormous numbers of parameters to train
- Each pass of batch gradient descent has to backpropagate through many many huge layers.
- Expensive in processing power, in memory, and in time.

Instead, **parameter-efficient fine tuning** (PEFT)

- Efficiently select a subset of parameters to update when finetuning.
- E.g., freeze some of the parameters (don't change them),
- And only update some a few parameters.

LoRA (Low-Rank Adaptation)

- Transformers have many dense matrix multiply layers
 - Like $\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V$, $\mathbf{W}^O$ layers in attention
- Instead of updating these layers during finetuning,
 - Freeze these layers
 - Update a low-rank approximation with fewer parameters.

LoRA

- Consider a **matrix W** (shape $[N \times d]$) that needs to be updated during finetuning via gradient descent.
 - Normally updates are ΔW (shape $[N \times d]$)
- In LoRA, we freeze W and update instead a low-rank decomposition of W :
 - A of shape $[N \times r]$,
 - B of shape $[r \times d]$, r is very small (like 1 or 2)
 - That is, during finetuning we update A and B instead of W .
 - Replace $W + \Delta W$ with $W + BA$.

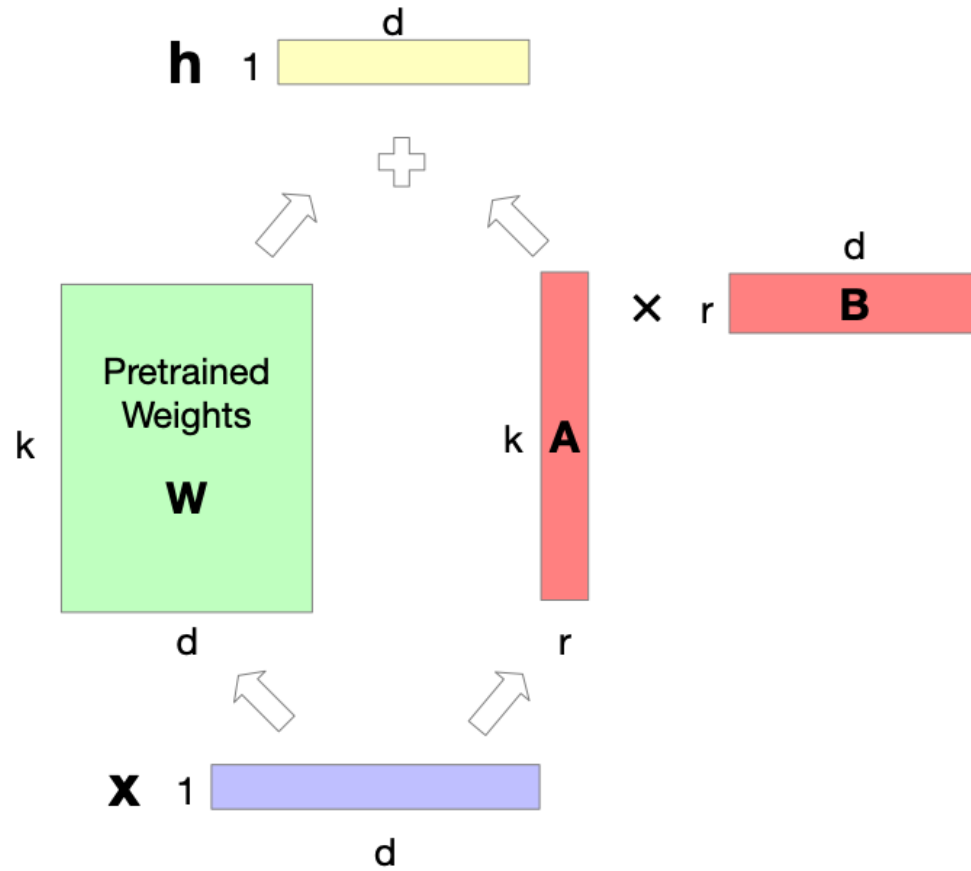
Forward pass: instead of

$$\mathbf{h} = \mathbf{xW}$$

We do

$$\mathbf{h} = \mathbf{xW} + \mathbf{xAB}$$

LoRA



Large Language Models

Dealing with Scale